U.PORTO
FACULDADE DE CIÊNCIAS
UNIVERSIDADE DO PORTO

Performance Analysis Report

Large Scale Data Science
[UC] CC3047

Inês Neves ---------- up202206252
Maria Beatriz Moreira ---------- up202208293

Table of Contents

1. **Introduction**
   - 1.1 Brief Background on PySpark, Dask, Modin, JobLib, RAPIDS, and Koalas
   - 1.2 Project Goals

2. **Materials and Methods**
   - 2.1 Machines Used and Their Characteristics
   - 2.2 Datasets Description

3. **Experiments**
   - 3.1 Experiment 1: Replication of NYC Taxi Driver Dataset Study
     - 3.1.1 Objective
     - 3.1.2 Execution Steps
     - 3.1.3 Execution Time Comparisons
   - 3.2 Experiment 2: Performance Benchmarking Across Library Combinations
     - 3.2.1 Objective
     - 3.2.2 Configuration Setups
       - – Dask + Modin
       - – Dask + RAPIDS
       - – Joblib
     - 3.2.3 Results and Performance Analysis

4. **Machine Learning Analysis**
   - 4.1 Methodology
   - 4.2 Results and Analysis

5. **Discussion and Conclusions**

1. Introduction

Efficient data processing has become a cornerstone of modern data science, particularly in scenarios involving large-scale datasets and machine learning (ML) workflows. With the growing volume and complexity of data, selecting the appropriate Python library can have a significant impact on computational performance, memory usage, and overall development efficiency.

In this context, this project evaluates the functionality and performance of several Python libraries tailored for big data processing: **PySpark**, **Dask**, **Modin**, **Joblib**, **RAPIDS**, and **Koalas**. Each of these tools offers distinct approaches to parallelism and scalability, ranging from distributed computing frameworks to GPU-accelerated processing. An overview of the Python libraries previously introduced is presented in the following section.

1.1. Libraries Overview/Description:

The Python API for Apache Spark, a distributed computing framework designed for large-scale data processing. It provides two main components: Resilient Distributed Datasets (RDDs), which enable fault-tolerant parallel processing, and DataFrames, which simplify operations on structured data. PySpark also includes MLlib, a ML library that supports scalable algorithms for tasks like classification, regression, and clustering.

A parallel computing library that scales Python code from a single machine to large clusters. It is built to integrate with popular data science libraries like pandas, NumPy, and scikit-learn, enabling operations on large datasets that do not fit in memory. Dask achieves parallelism through task scheduling and dynamic execution, allowing users to handle distributed data processing seamlessly.

A high-performance library that accelerates pandas by parallelizing operations across all available CPU cores. It serves as a drop-in replacement for pandas, with no need for significant code changes. Modin supports distributed execution through backends such as Dask or Ray, providing substantial speed improvements for large-scale data manipulation while maintaining pandas' compatibility.

A library designed for parallel processing and efficient caching of function outputs. It is commonly used with ML libraries, such as scikit-learn, to speed up tasks like model training and hyperparameter tuning. Joblib leverages Python's multiprocessing module and is ideal for parallelizing independent tasks, but it is not designed for distributed computing across multiple machines.

RAPIDS

A suite of open-source Python libraries developed by NVIDIA that leverages GPU acceleration for data science and ML workflows. RAPIDS provides GPU-accelerated equivalents of pandas (cuDF) and scikit-learn (cuML), allowing for high-speed data manipulation and model training. By offloading computations to the GPU, RAPIDS achieves significant performance gains for large datasets and complex ML tasks.

Koalas

Provides a pandas-like API on top of Apache Spark. It allows users to scale their pandas code for distributed computing environments without major modifications. Now integrated into PySpark as pyspark.pandas, Koalas simplifies the transition to distributed data processing, enabling pandas-style operations on large datasets using Spark's underlying distributed architecture.

1.2. Project Goals

This project explores the execution and performance of a complete ML pipeline using several Python libraries designed for big data processing.

The primary objectives are to:

- Identify performance bottlenecks associated with each specific library.
- Examine the syntactic and functional differences between the libraries.
- Determine which operations are best suited to one particular library.
- Evaluate the compatibility of these libraries with core Python packages such as pandas, NumPy, and scikit-learn.

To achieve these goals, a comparative study will be conducted between Koalas (PySpark) and Dask using the **NYC Taxi Trips** dataset, focusing on measuring execution times for defined operations. The study will document comparisons of performance for various data manipulation tasks, such as reading, aggregating, and transforming data. Subsequently, the code will be modified to include the libraries Modin, JobLib, and RAPIDS, using the same dataset, and expanding the analysis, to observe differences in performance as the data volume varies.

In a later phase of the project, a machine learning pipeline will be developed using the same dataset, including stages for data reading, preprocessing, model training, and validation.

The primary goal is to predict the target variable **"fare_amount"** using XGBRegressor and LogisticRegression models.

The performance of the models will be evaluated using metrics such as accuracy, precision, recall, and f1-score to understand how each library impacts the training and prediction processes.

2. Materials and Methods

2.1. Machines Used and Their Characteristics

All experiments in this project were conducted on single-machine environments, selected to balance performance, flexibility, and available cloud resources. No distributed processing across multiple nodes was performed. The computing environments consisted of a Vertex AI Workbench user-managed VM on Google Cloud Platform (GCP) for CPU-intensive experiments, and a local machine for GPU-accelerated workloads.

Cloud VM – Vertex AI Workbench (CPU Workloads)

A user-managed Vertex AI Workbench notebook instance was deployed on Google Cloud Platform to serve as the primary environment for data processing and benchmarking libraries such as Koalas (PySpark), Modin, Dask, and Joblib, all in single-node mode. This setup offered the flexibility of a managed notebook interface with the ability to customize the virtual machine backend.

VM Configuration:

- **Provisioning Platform:** Vertex AI Notebooks (User-managed)
- **Machine Type:** n2d-highmem-8 (8 vCPUs, 32 GB RAM)
- **CPU Platform:** AMD EPYC Milan
- **Boot Disk:** 500 GB Balanced Persistent Disk
- **Data Disk:** 100 GB Balanced Persistent Disk
- **Operating System:** Ubuntu 20.04 LTS
- **Python Version:** 3.10
- **Firewall Access:** HTTP and HTTPS enabled
- **External IP:** Ephemeral

Despite being inspired by Databricks' benchmark setup, full multi-node distributed experimentation (e.g., across multiple i3.xlarge-like machines) was not feasible due to cloud resource constraints.

Local Workstation – GPU Workloads

Due to limitations in Google Cloud Platform's free tier credits, GPU-based workloads could not be executed in the cloud environment. To overcome this, a personal workstation was used exclusively for experiments requiring GPU acceleration, particularly those involving the RAPIDS suite (e.g., cuDF, cuML) and GPU-enabled Dask pipelines.

Since RAPIDS does not support native Windows installations, the experiments were conducted within a WSL2 (Windows Subsystem for Linux 2) environment using a Miniforge-based Conda setup. This allowed the use of a compatible CUDA environment while leveraging the local NVIDIA GPU.

Hardware & Software Configuration:

- **Operating System:** Windows 11 (Host) with Ubuntu (WSL2 guest)
- **Processor:** Intel Core i7-12700H, 12th Gen, 2.30 GHz (14 cores, 20 threads)

- **Installed RAM:** 16 GB DDR5
- **GPU:** NVIDIA GeForce RTX 3060 Laptop GPU (6 GB VRAM)
- **CUDA Support:** Enabled via WSL2 with NVIDIA drivers and CUDA Toolkit 11.x
- **Environment:** Miniforge (Conda) on Ubuntu 22.04 (WSL2)
- **Python Version:** 3.10
- **System Type:** 64-bit, x64-based processor

This local setup was used exclusively for executing GPU-accelerated workloads that could not be run in the cloud environment. Specifically, it supported experiments involving RAPIDS libraries such as cuDF, cuML, and cuGraph, as well as Dask with CUDA backends and full machine learning pipelines requiring GPU support.

Despite having lower memory capacity compared to the GCP VM, the presence of a dedicated GPU made this setup well-suited for evaluating GPU-accelerated performance in large-scale data processing scenarios.

2.2. Datasets Description

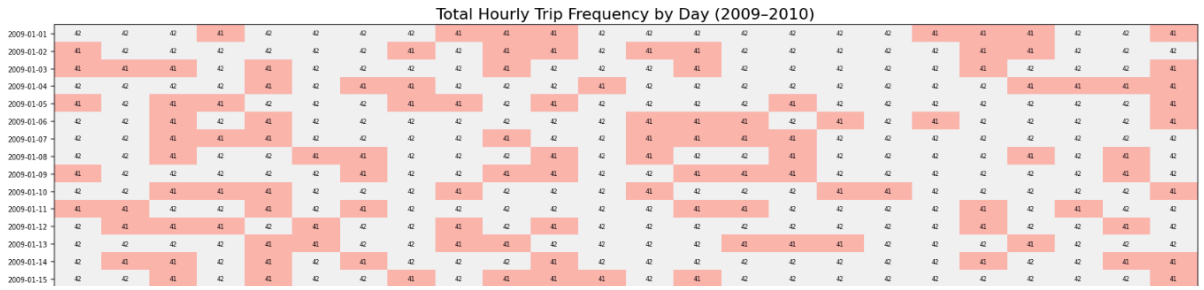
This project utilizes **three derived datasets** created from the publicly available NYC Yellow Taxi trip records. The original dataset consists of monthly parquet files ranging from January 2009 to December 2013, sourced from the **NYC Taxi and Limousine Commission (TLC).** However, due to format consistency and compatibility constraints, only records from 2009 and 2010 were used in this study.

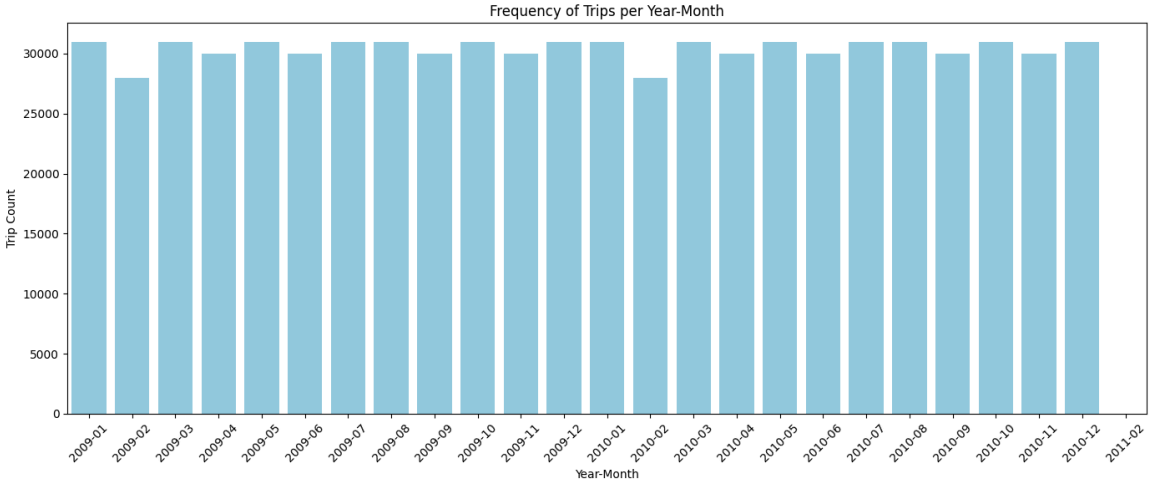
Each monthly file includes millions of taxi trip records, containing attributes such as timestamps for pickup and dropoff, geospatial coordinates, fare components, and passenger counts. To ensure consistency and usability across time, a standardization step was performed. This process involved renaming columns and aligning the schema across all source files to a unified format.

Following standardization, a stratified and balanced sampling strategy was applied. This method ensured an even distribution of samples across both days and hours, preserving temporal diversity and minimizing sampling bias. The result was the creation of three curated datasets at varying sizes, used to benchmark and compare the performance of different large-scale data processing libraries.

To validate the representativeness of the sampled datasets, heatmaps were generated to visualize the distribution of taxi trip records across hours of the day and days of the year. These plots confirm a consistent and balanced temporal spread over the two-year period used in the experiments. Due to the size of the full heatmaps, only the initial portion is shown here to highlight the uniformity achieved through the stratified sampling strategy. Additionally, bar plots displaying the number of samples per month are included to confirm that monthly representation is balanced across the dataset.

Small Dataset

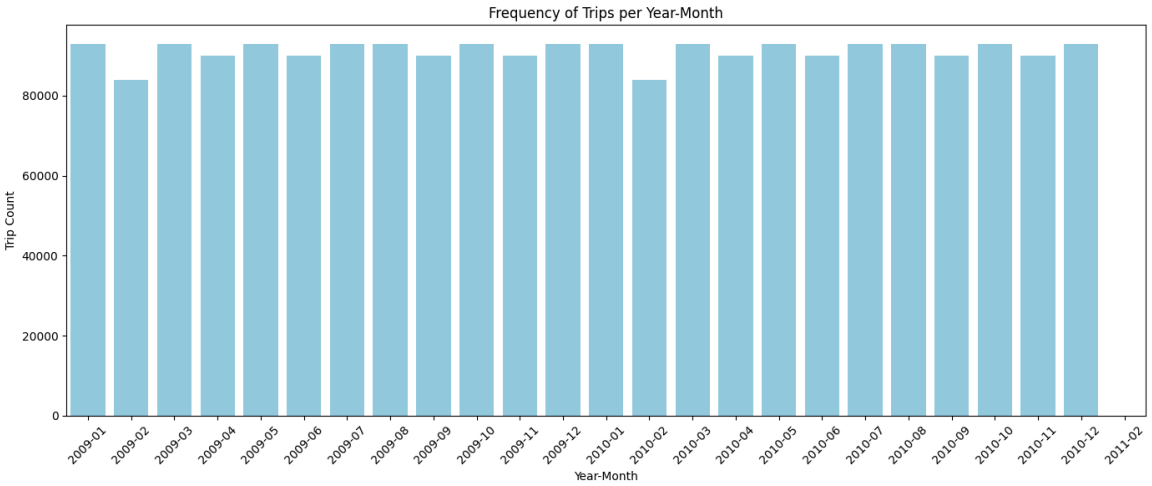




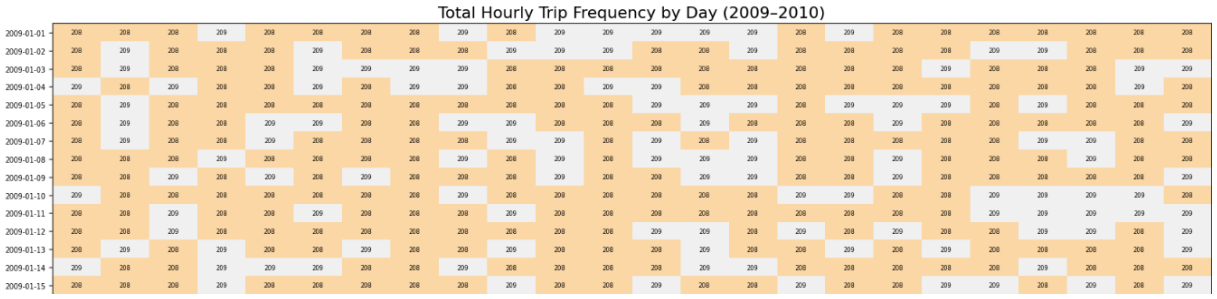
Medium Dataset

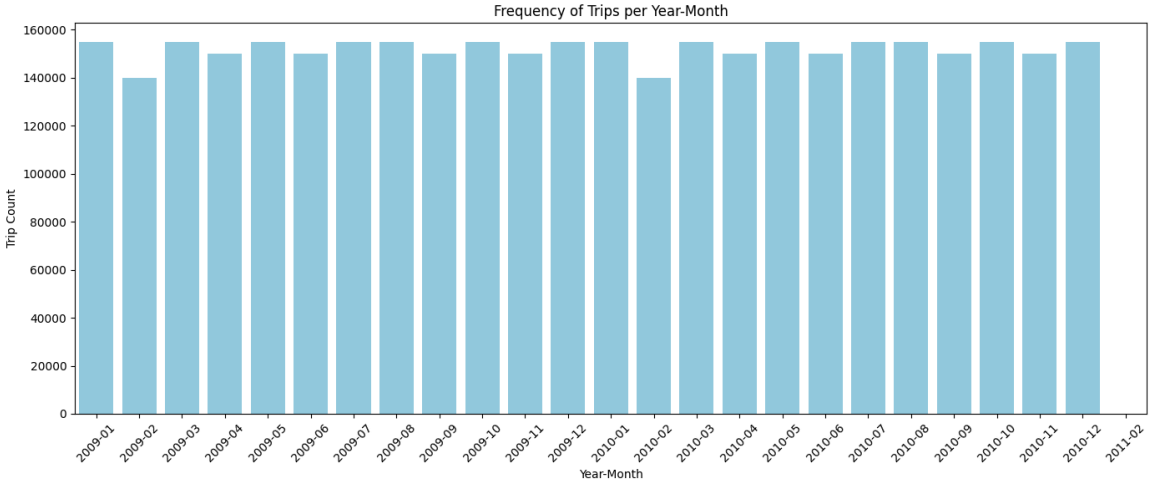
Total Hourly Trip Frequency by Day (2009–2010)

2009-01-01	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125
2009-01-02	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125
2009-01-03	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125
2009-01-04	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125
2009-01-05	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125
2009-01-06	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125
2009-01-07	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125
2009-01-08	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125
2009-01-09	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125
2009-01-10	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125
2009-01-11	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125
2009-01-12	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125
2009-01-13	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125
2009-01-14	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125
2009-01-15	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125	125



Large Dataset





Datasets

Dataset	Samples per Day	Total Records	Approximate Size
Small	1 000	730 009	105.8+ MB
Medium	3 000	2 190 009	317.5+ MB
Large	5 000	3 650 008	529.1+ MB

Each dataset shares an identical schema with 19 columns.
The full column structure is summarized below:

Column Name	Description	Data Type
vendor_name	Taxi vendor identifier	object
Trip_Pickup_DateTime	Timestamp of trip start	datetime64[ns]
Trip_Dropoff_DateTime	Timestamp of trip end	object
Passenger_Count	Number of passengers	int64
Trip_Distance	Trip distance in miles	float64
Start_Lon	Pickup location: longitude	float64

Start_Lat	Pickup location: latitude	float64
End_Lon	Dropoff location: longitude	float64
End_Lat	Dropoff location: latitude	float64
Payment_Type	Method of payment	object
Fare_Amt	Base fare amount	float64
surcharge	Additional charge applied to the fare	float64
Tip_Amt	Tip amount	float64
Tolls_Amt	Total tolls charged	float64
Total_Amt	Total transaction amount (fare + extras)	float64
year	Extracted from Trip_Pickup_DateTime	int64
month		int64
day		int64
hour		int64

3. Experiments

3.1. Experiment 1: Replication of NYC Taxi Driver Dataset Study

3.1.1. Objective

The goal of this experiment is to replicate and benchmark the performance of key data transformation operations outlined in a Databricks blog post, using Koalas (PySpark backend) and Dask. All experiments were conducted in a controlled, single-node CPU environment using the datasets described in Section 2.2. This experiment aims to provide insight into how each framework scales across varying dataset sizes and how efficiently they handle typical analytical workloads.

3.1.2 Execution Steps

Configuration Setup: Koalas

Environment and Library Configuration

Koalas was used as a high-level API to write pandas-like code while utilizing Apache Spark's distributed engine under the hood. For Koalas to work correctly, the Python environment must be consistent between the Spark driver and executors.

To enforce this, the following Spark and system-level configurations were applied:

```python
os.environ["PYSPARK_PYTHON"] = "/opt/conda/envs/cdle/bin/python3.7"
os.environ["PYSPARK_DRIVER_PYTHON"] = "/opt/conda/envs/cdle/bin/python3.7"
os.environ["JAVA_HOME"] = "/usr/lib/jvm/java-11-openjdk-amd64"
```

The Spark session was initialized with matching Python paths for both driver and executors to ensure compatibility:

```python
spark = SparkSession.builder \
    .appName("project") \
    .config("spark.pyspark.python", "/usr/bin/python3.7") \
    .config("spark.pyspark.driver.python", "/usr/bin/python3.7") \
    .getOrCreate()
```

Additionally, internal Spark caching was disabled to benchmark raw performance:

```python
spark.conf.set("spark.databricks.io.cache.enabled", "false")
```

This environment setup ensured that Koalas computations were backed by a Spark engine configured for consistency, transparency, and repeatability in performance testing.

Dataset Handling and Transformation

The benchmarking process for Koalas followed a three-stage experimental structure: standard, filtered, and cached workloads.

Data ingestion began with loading a Parquet file into a Koalas DataFrame:

```python
koalas_data = ks.read_parquet("Samples/sample_2009_2010_small_clean.parquet")
```

A subset of the data was created using a pandas-like boolean filter on the Tip_Amt column, selecting rows within a specific value range:

```python
expr_filter = (koalas_data.Tip_Amt >= 1) & (koalas_data.Tip_Amt <= 5)
```

```
koalas_filtered = koalas_data[expr_filter]
```

For the cached execution scenario, Koalas' integration with the Spark execution engine was leveraged to persist the filtered dataset in memory:

```python
koalas_filtered = koalas_filtered.spark.cache()
```

This ensured that subsequent operations could benefit from in-memory execution, minimizing disk I/O and recomputation.

To support more complex join-based operations, additional statistics were computed using groupby aggregations. The results were converted to a standard Koalas DataFrame with flattened column names for compatibility:

```python
other = ks.DataFrame(groupby_statistics(koalas_data).to_pandas())
other.columns = pd.Index([e[0]+'_' + e[1] for e in other.columns.tolist()])
```

Configuration Setup: Dask

Environment and Library Configuration

Dask was used as a native Python parallel computing library to enable scalable analytics across a cluster. It operates by distributing tasks via a Dask distributed client, which was initialized before executing benchmark workloads:

```python
from dask.distributed import Client
client = Client()
```

This setup allowed the orchestration of parallel computations across CPU cores or nodes in a Dask cluster. Dask was paired with:

- **Pandas:** For final result formatting
- **NumPy:** For vectorized math operations
- **Parquet:** As the storage format for data ingestion

All transformations and statistical operations were lazily evaluated and explicitly triggered using *.compute().*

Dataset Handling and Transformation

Data ingestion was performed via Dask's native Parquet reader, which returned a lazy Dask DataFrame:

```python
dask_data = dd.read_parquet("Samples/sample_2009_2010_small_clean.parquet")
```

A filtered subset was created with boolean indexing, selecting rows with Tip_Amt values between 1 and 5:

```python
expr_filter = (dask_data.Tip_Amt >= 1) & (dask_data.Tip_Amt <= 5)
dask_filtered = dask_data[expr_filter]
```

To accelerate repeated computation, the filtered dataset was explicitly persisted in memory across the Dask cluster:

```python
dask_filtered = client.persist(dask_filtered)
wait(dask_filtered)
```

This represented the cached execution phase, where tasks were precomputed and stored to reduce latency in follow-up benchmarks.

Aggregated statistics (e.g., groupby-mean and standard deviation) were computed with *.agg()* and finalized using *.compute()*, ensuring full materialization of results. Additional join operations were supported by *dd.merge().*

To enable a fair and comprehensive comparison, three standardized and stratified datasets (Small, Medium, Large) were used. Each operation was executed using both Dask and Koalas, with timing captured via a custom benchmark() function. All experiments were executed on the same cloud-based VM instance (GCP Vertex AI Workbench, 8 vCPUs, 64 GB RAM) to maintain hardware consistency.

Benchmarked Operations

The following operations were selected to cover typical analytical workloads:

- **read file:** Load dataset from Parquet format
- **count:** Count number of rows
- **count index length:** Length of index (alternate row count)
- **mean:** Column-wise average computation
- **standard deviation:** Column-wise standard deviation
- **addition of columns:** Add two numeric columns
- **mean of columns addition:** Compute mean of sum of two columns
- **multiplication of columns:** Multiply two numeric columns
- **mean of columns multiplication:** Compute mean of product of two columns
- **value counts:** Frequency count of values in a column

- **complex arithmetic ops:** Series of chained arithmetic operations
- **mean of complex arithmetic ops:** Compute mean of result from complex arithmetic
- **groupby statistics:** Group by column and compute multiple statistics
- **join:** Perform join with auxiliary DataFrame
- **join count:** Count rows after join operation

Each operation was executed using both frameworks, and results were stored in structured logs to support later comparative analysis.

3.1.3 Execution Time Comparisons

Small Dataset

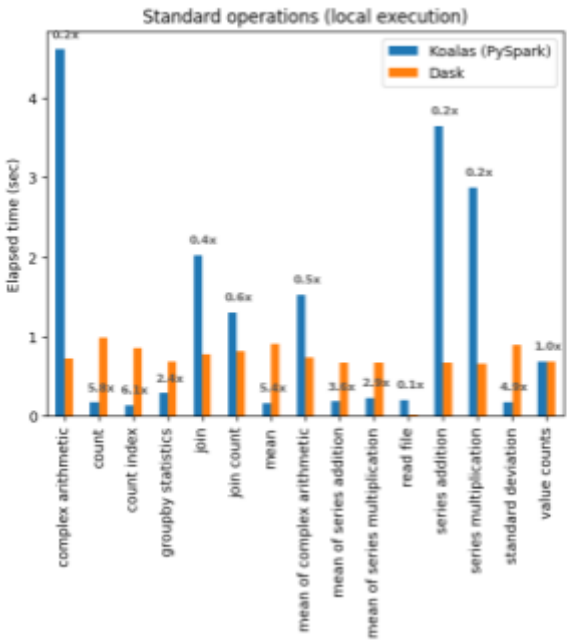
The following section presents detailed execution time comparisons between Koalas (PySpark) and Dask when applied to the **small dataset** across three processing configurations: standard operations, operations with filtering, and operations with both filtering and caching.

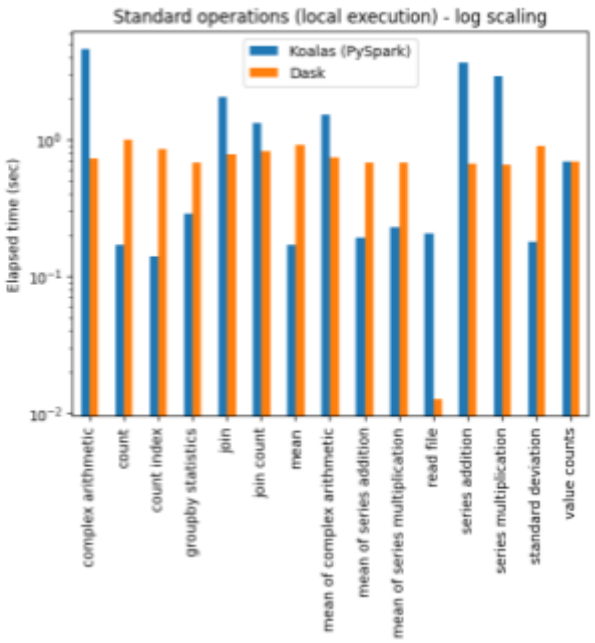
```python
Standard Operations Summary:
                              Koalas (PySpark)      Dask
read file                             0.206737  0.012752
count                                 0.169843  0.992259
count index                           0.139787  0.853722
mean                                  0.168816  0.908047
standard deviation                    0.179950  0.890122
mean of series addition               0.190095  0.679281
series addition                       3.651553  0.670495
mean of series multiplication         0.230027  0.676225
series multiplication                 2.879629  0.658181
value counts                          0.689393  0.686443
complex arithmetic                    4.610354  0.728920
mean of complex arithmetic            1.529027  0.743922
groupby statistics                    0.288363  0.681104
join count                            1.308148  0.819171
join                                  2.027326  0.773804
Operations with Filtering Summary:

                              Koalas (PySpark)      Dask
read file                             0.142224  0.013402
count                                 0.192469  0.979224
count index                           0.120785  0.840879
mean                                  0.205450  0.930835
standard deviation                    0.241870  0.900554
mean of series addition               0.251020  0.724663
series addition                       1.245989  0.694898
mean of series multiplication         0.201328  0.719771
series multiplication                 1.190104  0.670101
mean of complex arithmetic            1.368959  0.782910
complex arithmetic                    2.478188  0.735402
value counts                          0.430056  0.697358
groupby statistics                    0.359634  0.718106
join                                  1.924822  0.772424
join count                            0.755365  0.744931


Operations with Filtering and Caching Summary:

                              Koalas (PySpark)      Dask
read file                             0.143257  0.013085
```

```
count                                 0.038452  0.219949
count index                           0.046144  0.022429
mean                                  0.100491  0.042456
standard deviation                    0.089760  0.041414
mean of series addition               0.102020  0.032941
series addition                       0.900061  0.023919
mean of series mu
ltiplication          0.097529  0.035269
series multiplication                 0.737541  0.016406
value counts                          0.141458  0.025166
complex arithmetic                    1.916884  0.059453
mean of complex arithmetic            1.214483  0.080001
groupby statistics                    0.301568  0.059836
join count                            0.368979  0.270606
join                                  0.578082  0.264318
```

Given the results obtained in the **Koalas_Dask_S.ipynb notebook**, we elaborated the following analysis for the standard operations, operations with filtering and operations with filtering and cache.

Standard Operations





<u>Analysis:</u>

- **Overall trends:**

Dask outperforms Koalas (PySpark) in a few operations, often by a large margin, as indicated by the Dask / Koalas ratios well below 1. For instance, complex arithmetic operations and file reading show Dask being about 6 to 16 times faster than Koalas.

- **Operations where Dask excels:**

- Complex arithmetic: Dask is about 6.3x faster (Koalas: 4.61s vs Dask: 0.73s).

- File reading: Dask drastically outperforms Koalas by more than 16x, which is likely due to better I/O optimizations or parallelism in Dask.
- Series addition and multiplication: Dask is roughly 4–5x faster.
- Count and mean operations: Dask is about 5–6x faster than Koalas, showing strong performance in these basic aggregation tasks.

- **Operations where Koalas is relatively faster or competitive:**

- For join and join count operations, Koalas is about 1.6 to 2.6 times faster than Dask. This suggests Koalas might have more optimized implementations for joins in this context.

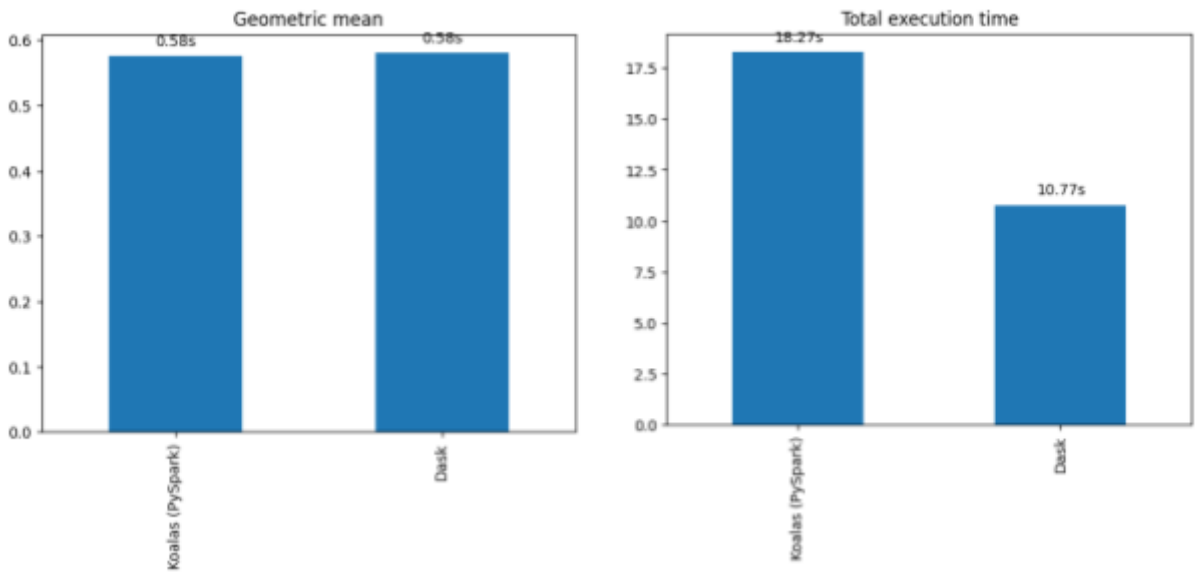
- For value counts, performance is roughly equal, with ratios close to 1, indicating similar efficiency.

- **Intermediate cases:**

- Groupby statistics and mean of series operations show Dask outperforming Koalas by 2–3 times, which is still significant but less extreme compared to other operations.

- **Interpretation of ratios:**
- Values less than 1 in Dask / Koalas mean Dask is faster.
- Values greater than 1 mean Koalas is faster.

Practical insight:
These results suggest that Dask is particularly efficient for arithmetic-heavy and aggregation tasks, as well as for reading files, possibly due to better distributed execution or lower overhead. Koalas may retain an edge for some relational operations like joins, where Spark's optimization strategies are well-established.



The total execution time with Dask was **10.77 seconds**, significantly lower than **18.27 seconds** for the other system, representing a performance improvement of approximately **41%**.

In the notebook, we also compare the following metrics:

- Performance diff % (simple avg): 2.28% — The average percent difference in performance is about 2.3%, indicating a moderate overall change.

- Performance diff % (geomean): 1.01% — The geometric mean difference is around 1%, showing most differences are small with some outliers raising the simple average.

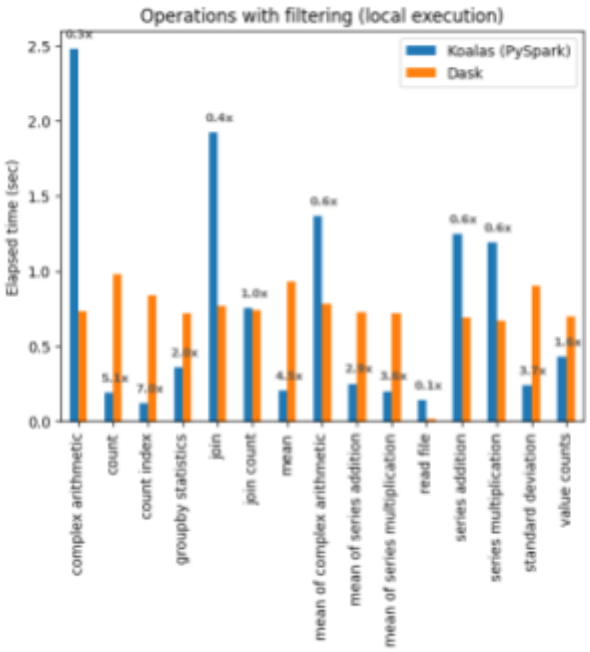
- Performance diff (arith) % (simple avg): 0.19% — Using an alternative arithmetic metric, the average difference is very small, suggesting near-equal performance.

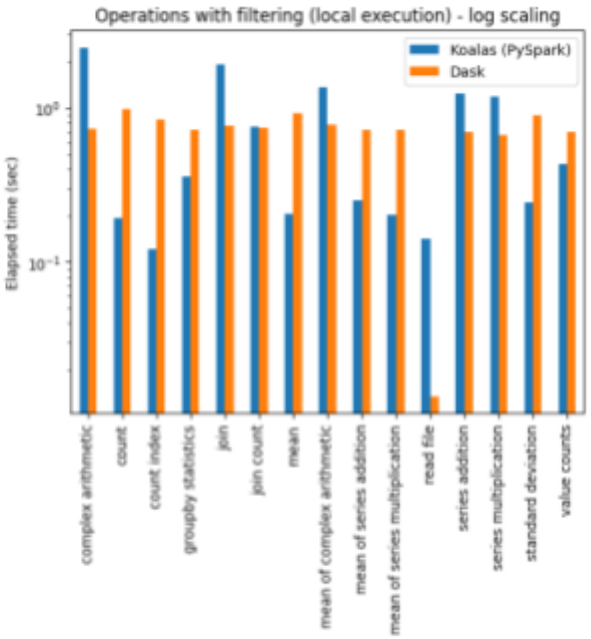
- Performance diff (arith) % (geomean): 0.19% — The geometric mean confirms that performance differences are minimal in this metric.

- Performance diff (basic stats) % (simple avg): 3.88% — This statistic-based metric shows higher variability, indicating occasional significant performance differences.

- Performance diff (basic stats) % (geomean): 2.47% — The geometric mean is lower, suggesting some extreme cases increase variability but the overall spread is moderate.

Operations with Filtering





Analysis

- **Overall trends:**

Dask outperforms Koalas in some operations, often by a significant margin, as shown by many Dask / Koalas ratios well below 1. However, for certain aggregation operations like count and mean, Koalas shows faster execution times.

- **Operations where Dask excels:**

- Complex arithmetic: Dask is about 3.4x faster (Koalas: 2.48s vs Dask: 0.74s).
- File reading: Dask drastically outperforms Koalas by more than 10x, indicating superior I/O efficiency.
- Join: Dask is about 2.5x faster (Koalas: 1.92s vs Dask: 0.77s).
- Mean of complex arithmetic: Dask is about 1.75x faster.
- Series addition and multiplication: Dask is around 1.8x faster.

- **Operations where Koalas is relatively faster or competitive:**

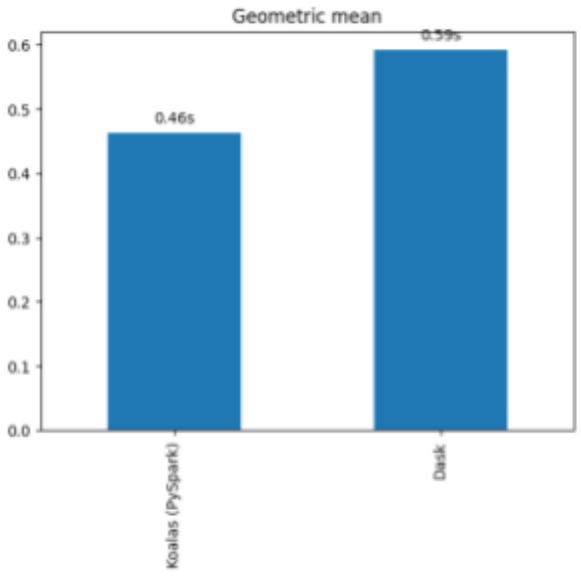
- For count and count index operations, Koalas is about 5 to 7 times faster than Dask, showing a clear advantage in simple counting aggregations.
- For mean, Koalas is roughly 4.5x faster than Dask.
- For join count, performance is nearly identical, with ratios close to 1.

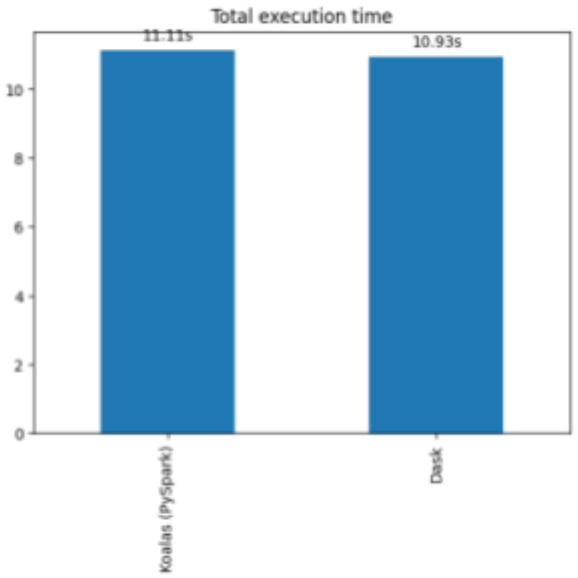
- **Intermediate cases:**

Groupby statistics and value counts show Dask being 1.6 to 2 times slower than Koalas, but still within a moderate performance gap.

- **Interpretation of ratios:**
- Values less than 1 in Dask / Koalas mean Dask is faster.
- Values greater than 1 mean Koalas is faster.

Practical insight:
These results suggest Dask is particularly efficient for arithmetic-heavy operations, file reading, and joins, likely benefiting from optimized parallel execution and I/O. Conversely, Koalas (PySpark) excels at simple counting and mean aggregation tasks, potentially due to Spark's mature optimization for these operations. Thus, choice of framework may depend on the workload focus—Dask for complex computations and I/O, Koalas for fast aggregations.





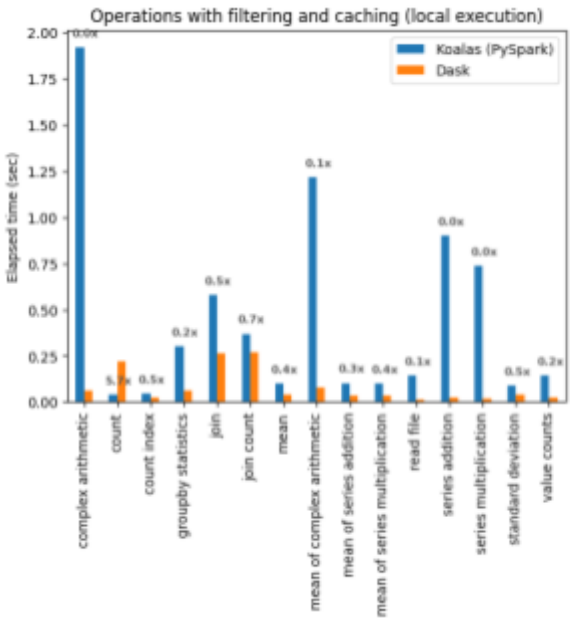
The total execution time for filtering operations was **10.93 seconds** with Dask and **11.11 seconds** with Koalas, showing nearly identical performance with Dask being only slightly faster (about **1.6%** improvement).

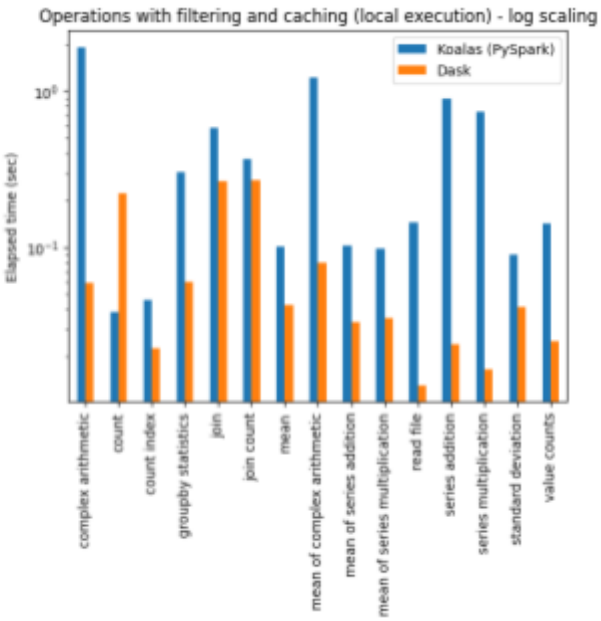
In the notebook, we also compare the following metrics:

- Performance diff % (simple avg): 2.26% — The average percentage difference in performance is around 2.3%, indicating a moderate overall shift across operations.

- Performance diff % (geomean): 1.28% — The geometric mean of the performance differences is approximately 1.28%, suggesting that most performance gaps are small, with a few outliers slightly increasing the average.

Operations with Filtering and Caching





Analysis:

- **Overall trends:**

Dask significantly outperforms Koalas in most operations, with some speedups exceeding 30–40x, especially in arithmetic-heavy and file-based tasks. Koalas, however, retains a slight edge in a few aggregation operations like `count`.

- **Operations where Dask excels:**

- Series multiplication: Dask is ~45x faster (Koalas: 0.74s vs Dask: 0.016s).
- Series addition: Dask is ~38x faster (Koalas: 0.90s vs Dask: 0.024s).
- Complex arithmetic: Dask is ~32x faster (Koalas: 1.92s vs Dask: 0.059s).
- Mean of complex arithmetic: Dask is ~15x faster.
- Read file: Dask is ~11x faster.
- Groupby statistics & value counts: Dask is ~5x faster, showing strong performance in grouped operations.

- **Operations where Koalas is relatively faster or competitive:**

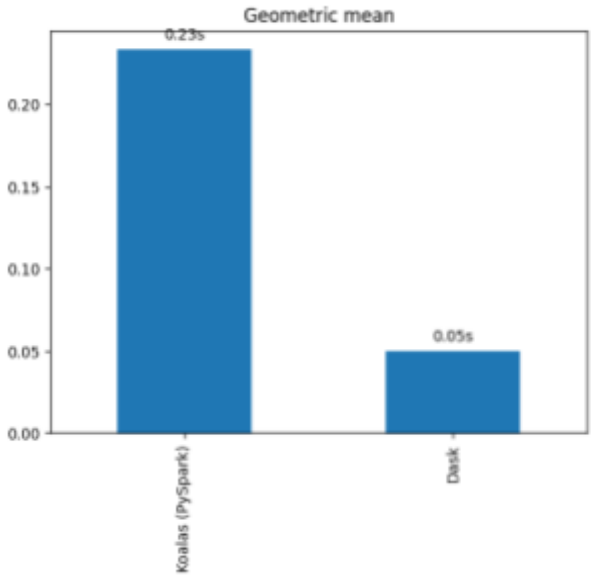
- Count: Koalas is ~5.7x faster (Dask: 0.22s vs Koalas: 0.038s).
- Count index: Koalas is ~2x faster.
- Join count & standard deviation: Dask is slightly slower, with ratios around 0.73–0.46, indicating modest advantage for Koalas.

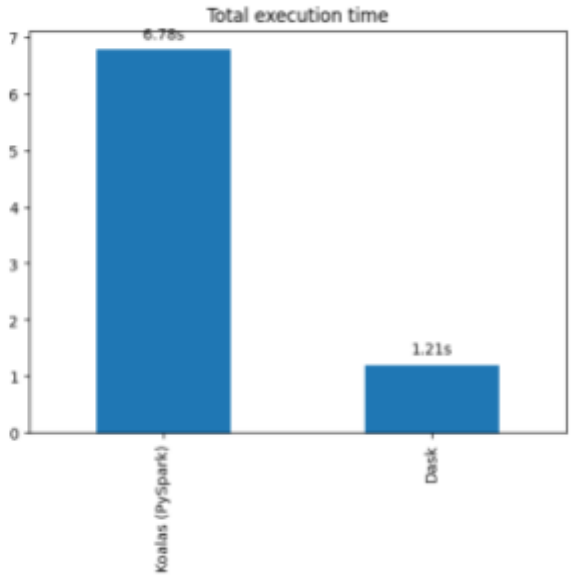
- **Intermediate cases:**
- Join, mean, and mean of series operations show Dask being ~2–3x faster, suggesting consistent, moderate advantages.
- **Interpretation of ratios:**

- Values less than 1 in Dask / Koalas indicate Dask is faster.
- Values greater than 1 suggest Koalas is faster.

Practical insight:

These results confirm Dask's strong advantage for arithmetic-heavy operations, file reading, and grouped computations, likely due to efficient parallel execution and minimal overhead. Koalas, while generally slower, shows solid performance in basic aggregations like count and index operations, potentially benefiting from Spark's internal optimizations. For workloads with complex math and large I/O, Dask is clearly preferable.





The total execution time for operations with filtering and cache was **1.21 seconds** with Dask and **6.78 seconds** with Koalas, meaning Dask was approximately **5.6 times faster**, demonstrating a significant performance advantage.

In the notebook, we also compare the following metrics:

- Performance diff % (simple avg): 0.64% — The average percentage performance difference is low, indicating nearly equivalent performance between methods overall.

- Performance diff % (geomean): 0.21% — The geometric mean confirms minimal variation, suggesting consistent performance with very small gaps.

- Performance diff % (count, simple avg): 2.31% — When considering only count-related operations, the average performance difference rises to around 2.3%, implying slightly more variation in this subset.
- Performance diff % (count, geomean): 1.27% — The geometric mean for count operations also shows moderate variation, suggesting a few operations may be skewing the average.

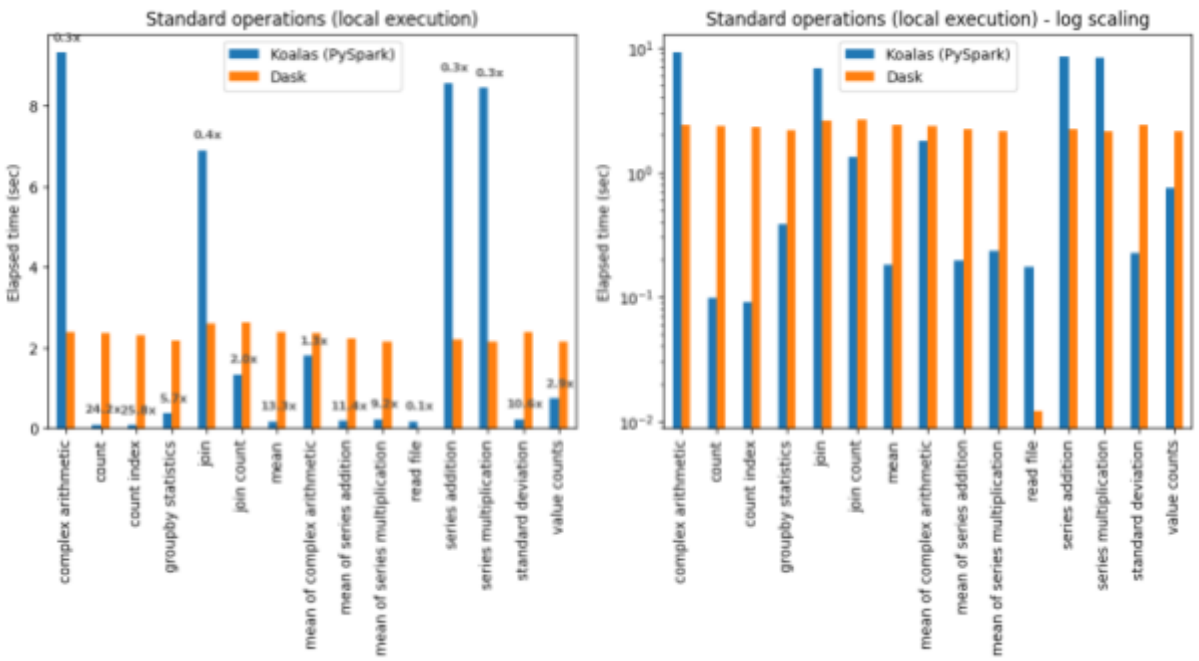
Medium Dataset

The following section presents detailed execution time comparisons between Koalas (PySpark) and Dask when applied to the **medium dataset** across three processing configurations: standard operations, operations with filtering, and operations with both filtering and caching.

```python
Standard Operations Summary:
                                Koalas (PySpark)      Dask
read file                               0.175478  0.012189
count                                   0.097523  2.355766
count index                             0.090070  2.326114
mean                                    0.180760  2.395913
standard deviation                      0.226536  2.393106
mean of series addition                 0.195258  2.227127
series addition                         8.567658  2.209179
mean of series multiplication           0.234717  2.157609
series multiplication                   8.457660  2.147549
value counts                            0.752996  2.151824
complex arithmetic                      9.319679  2.398656
mean of complex arithmetic              1.802734  2.377824
groupby statistics                      0.384018  2.186385
join count                              1.343497  2.641861
join                                    6.902165  2.616759


Operations with Filtering Summary:

                                Koalas (PySpark)      Dask
read file                               0.129181  0.011258
count                                   0.208609  2.444902
count index                             0.119544  2.428260
mean                                    0.228150  2.530257
standard deviation                      0.239427  2.413083
mean of series addition                 0.208720  2.246880
series addition                         2.719321  2.243072
mean of series multiplication           0.233332  2.265481
series multiplication                   3.484863  2.204679
mean of complex arithmetic              1.441590  2.296589
complex arithmetic                      4.869501  2.280747
value counts                            0.408881  2.222444
groupby statistics                      0.369355  2.268498
join                                    6.624686  2.272221
join count                              1.312817  2.326004


Operations with Filtering and Caching Summary:

                                Koalas (PySpark)      Dask
read file                               0.126777  0.010928
count                                   0.041290  0.214996
count index                             0.047761  0.017538
mean                                    0.126425  0.035154
standard deviation                      0.078255  0.038286
mean of series addition                 0.101795  0.034197
series addition                         1.938681  0.062088
mean of series multiplication           0.102574  0.033715
series multiplication                   2.140677  0.038474
value counts                            0.188008  0.027637
complex arithmetic                      3.233056  0.118194
mean of complex arithmetic              1.323034  0.115331
groupby statistics                      0.273932  0.072259
join count                              0.404148  0.289622
join                                    0.677275  0.275390
```

Given the results obtained in the **Koalas_Dask_M.ipynb notebook**, we elaborated the following analysis for the standard operations, operations with filtering and operations with filtering and cache.

Standard Operations



Analysis:

- **Overall trends:**

Dask significantly underperforms Koalas in most operations in this test, with several operations showing Dask to be over 10–25x slower. This is especially true for simple aggregations like `count`, `mean`, and `standard deviation`. Koalas, in contrast, shows superior performance across almost all operations, with the exception of `read file`, where Dask is notably faster.

- **Operations where Koalas excels:**

- Count: Koalas is ~24x faster (Koalas: 0.098s vs Dask: 2.36s).
- Count index: Koalas is ~26x faster (Koalas: 0.090s vs Dask: 2.33s).
- Standard deviation: Koalas is ~10.6x faster (Koalas: 0.227s vs Dask: 2.39s).
- Mean: Koalas is ~13x faster.
- Mean of series addition / multiplication: Koalas is ~9–11x faster.
- Groupby statistics: Koalas is ~5.7x faster.
- Value counts: Koalas is ~2.9x faster.

- **Operations where Dask is relatively faster or competitive:**

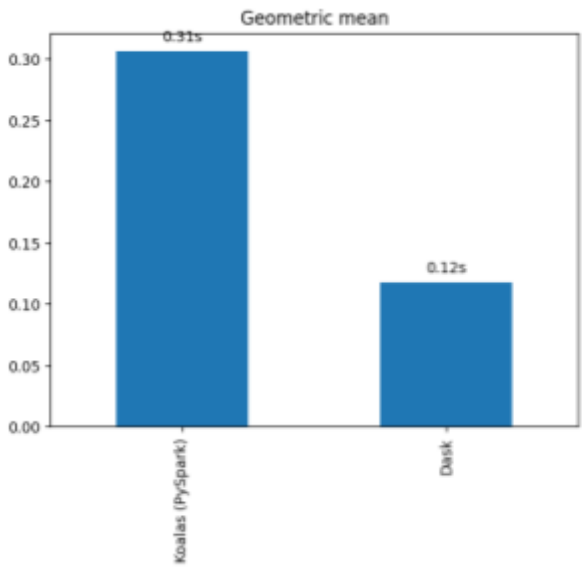
- Read file: Dask is ~14x faster (Dask: 0.012s vs Koalas: 0.175s), showing strong I/O efficiency.
- Join and complex arithmetic: Koalas is still faster (~3.8x), but the difference is smaller compared to aggregation operations.
- Join count & mean of complex arithmetic: Moderate differences, with Koalas around ~1.3–2x faster.

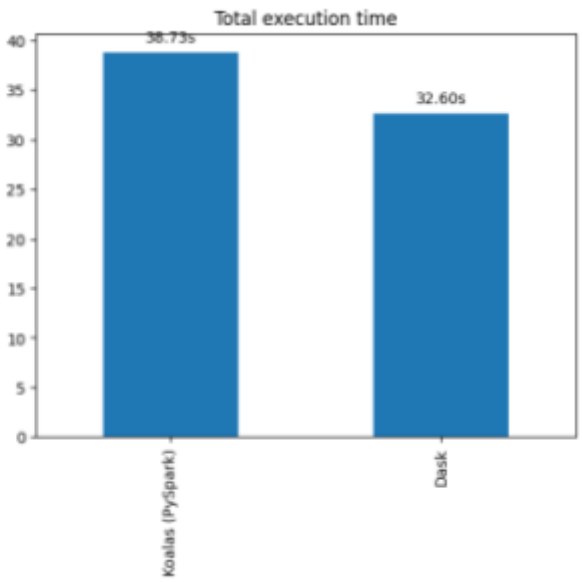
- **Interpretation of ratios:**

- Values less than 1 in Dask / Koalas indicate Dask is faster.
- Values greater than 1 suggest Koalas is faster.

- **Practical insight:**

These results reveal a notable shift compared to previous tests: Koalas outperforms Dask in most standard operations when no filtering or caching is applied. In this scenario, the Spark-based execution model of Koalas proves more efficient for basic computations and aggregations. The only consistent exception remains file reading, where Dask continues to excel. Thus, for unfiltered and uncached standard workloads, Koalas may provide superior performance in a wide range of operations.





The total execution time for operations with filtering and cache was 32.60 seconds with Dask and 38.73 seconds with Koalas, indicating that Dask was about 1.19 times faster, showing a modest performance advantage.

In the notebook, we also compare the following metrics:

- Performance diff % (simple avg): 7.16% — On average, performance differs by about 7.2% across all operations, indicating a moderate overall variation between the two frameworks.

- Performance diff % (geomean): 2.23% — The geometric mean shows a smaller difference, suggesting that most operations have minimal performance variation, with some outliers increasing the simple average.

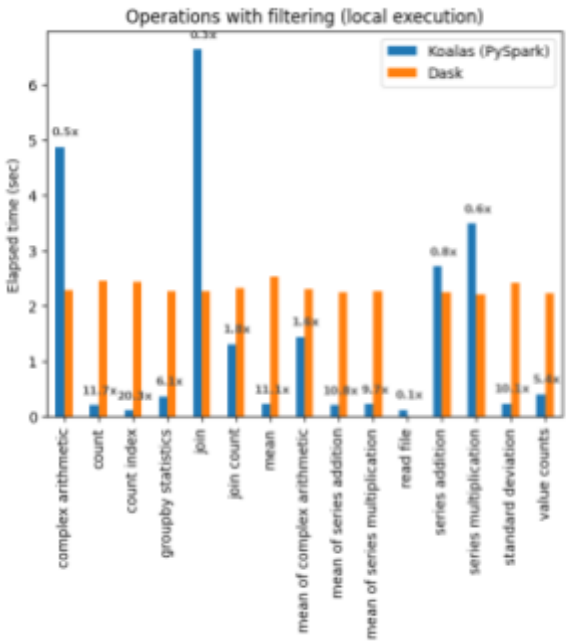
- Performance diff (arithmetic) % (simple avg): 0.26% — Arithmetic operations show very close performance between the frameworks, with only a quarter-percent average difference.

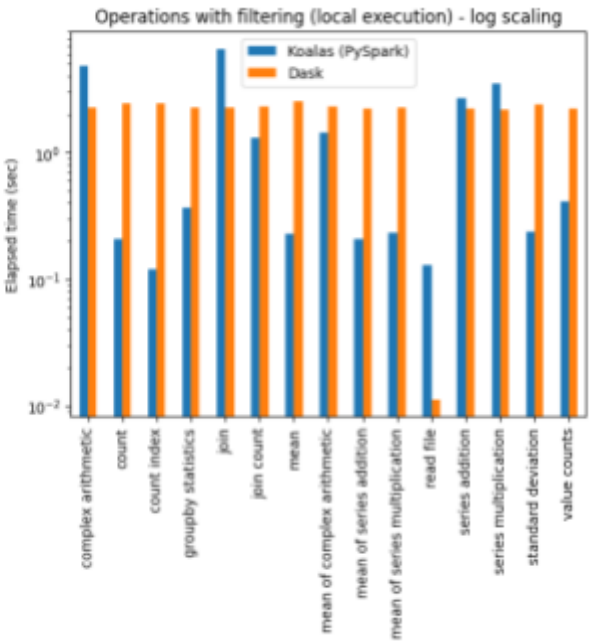
- Performance diff (arithmetic) % (geomean): 0.26% — The geometric mean confirms the tight performance gap in arithmetic tasks.

- Performance diff (basic stats) % (simple avg): 12.69% — Basic statistical operations exhibit the largest average difference, with performance varying by nearly 13%.

- Performance diff (basic stats) % (geomean): 6.34% — The geometric mean for basic stats indicates moderate variation, showing that some basic stats operations are notably different in performance while others are closer.

Operations with Filtering





<u>Analysis:</u>

- **Overall trends:**

Koalas consistently outperforms Dask in a majority of operations in this test scenario, particularly in aggregation and arithmetic operations. Dask only shows marginal advantages in more complex arithmetic tasks and maintains its lead in file reading performance.

- **Operations where Koalas excels:**

- Count: Koalas is ~11.7x faster (Koalas: 0.209s vs Dask: 2.44s).
- Count index: Koalas is ~20x faster (Koalas: 0.120s vs Dask: 2.43s).
- Mean: Koalas is ~11x faster.
- Standard deviation: Koalas is ~10x faster.
- Mean of series addition / multiplication: Koalas is ~9–10x faster.
- Groupby statistics: Koalas is ~6.1x faster.
- Value counts: Koalas is ~5.4x faster.

- **Operations where Dask is relatively faster or competitive:**

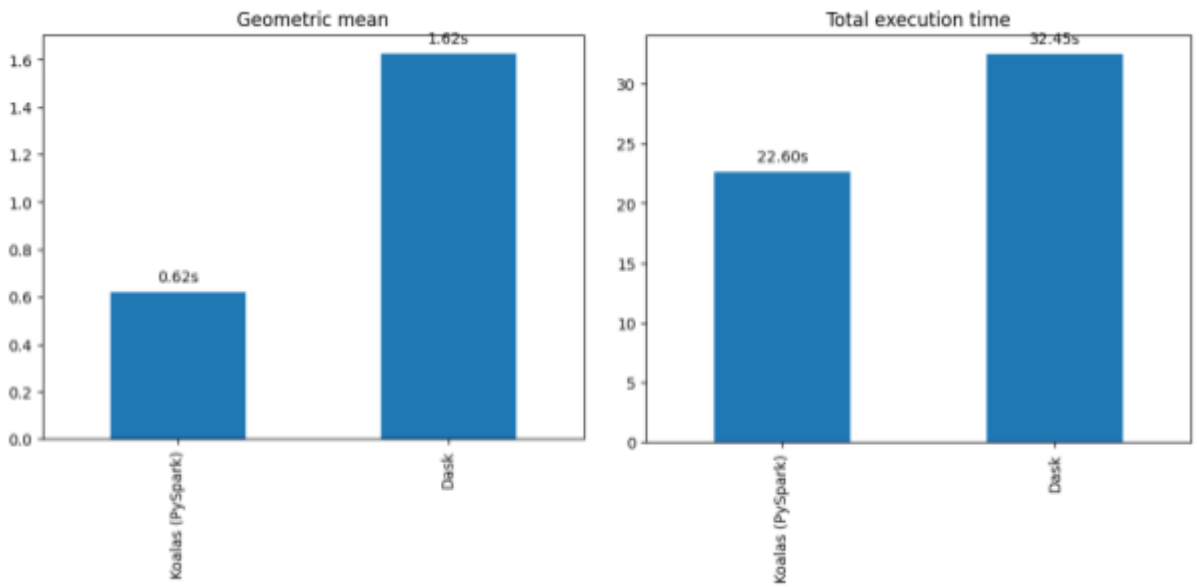
- Read file: Dask is ~11.5x faster (Dask: 0.011s vs Koalas: 0.129s).
- Complex arithmetic: Koalas is ~2.1x faster, showing smaller margin compared to other operations.
- Join and series operations: Koalas is moderately faster (~1.2x to ~2.9x), with Dask narrowing the performance gap.
- Join count and mean of complex arithmetic: Koalas maintains an advantage, though smaller (~1.6x–1.8x faster).

- **Interpretation of ratios:**

- Values less than 1 in Dask / Koalas indicate Dask is faster.
- Values greater than 1 suggest Koalas is faster.

Practical insight:

This comparison highlights Koalas' efficiency in arithmetic and aggregation-heavy workflows on filtered datasets without caching. Its consistent speed advantage over Dask—particularly in fundamental computations like counts, means, and standard deviations—demonstrates the strength of Spark's execution engine when handling filtered data. However, Dask continues to lead in I/O-bound tasks such as file reading and shows a reduced performance gap in joins and arithmetic operations. Koalas is the preferred choice for filtered analytical workloads without caching, while Dask remains well-suited for lightweight I/O-focused tasks.



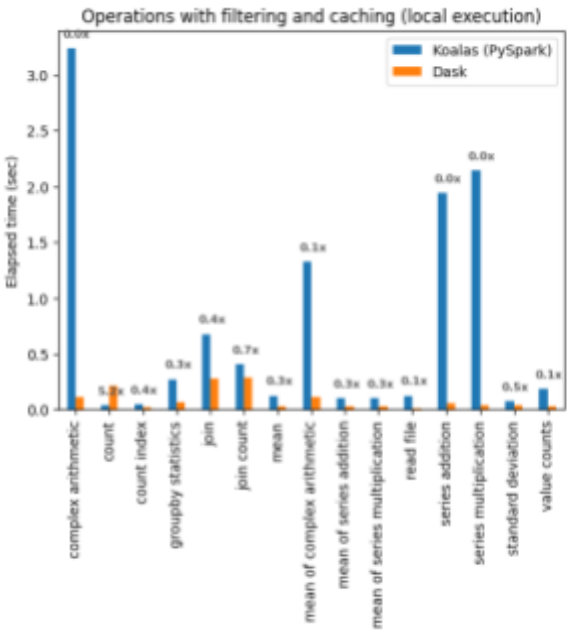
The total execution time for operations with filtering and cache was 32.45 seconds with Dask and 22.60 seconds with Koalas, indicating that Koalas was approximately 1.44 times faster, demonstrating a clear performance advantage.

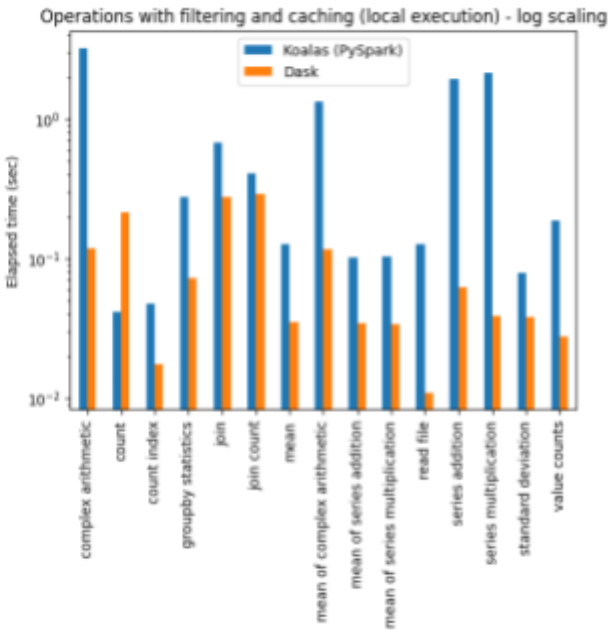
In the notebook, we also compare the following metrics:

- Performance diff % (simple avg): 6.06% — On average, performance differs by about 6% across all operations, indicating a moderate level of variation overall.

- Performance diff % (geomean): 2.62% — The geometric mean shows a smaller difference, suggesting that most operations have relatively close performance, with a few outliers increasing the simple average.

Operations with Filtering and Caching





<u>Analysis</u>

- **Overall trends:**

Dask significantly outperforms Koalas across nearly all operations in this cached and filtered scenario, often by large margins exceeding 20–50x in arithmetic-heavy tasks. Koalas only retains a moderate advantage in a few cases like count index and join count.

- **Operations where Dask excels:**

- Series multiplication: Dask is ~56x faster (Koalas: 2.14s vs Dask: 0.038s).
- Series addition: Dask is ~31x faster.

- Complex arithmetic: Dask is ~27x faster.
- Mean of complex arithmetic: Dask is ~11x faster.
- Read file: Dask is ~12x faster.
- Value counts: Dask is ~6.8x faster.
- Operations where Koalas is relatively faster or competitive:
- Count index: Koalas is ~2.7x faster.
- Join count: Koalas is ~1.4x faster.

- **Intermediate cases:**

Count, groupby statistics, join, mean, and standard deviation show Dask being about 2–4x faster, suggesting consistent advantage but with smaller gaps compared to arithmetic operations.

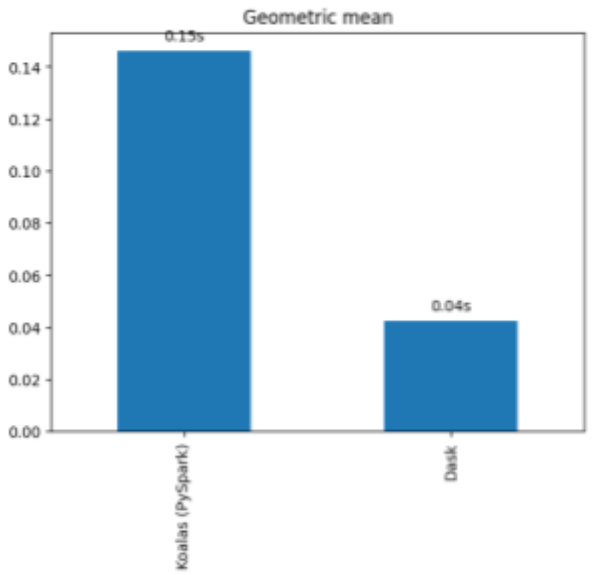
- **Interpretation of ratios:**

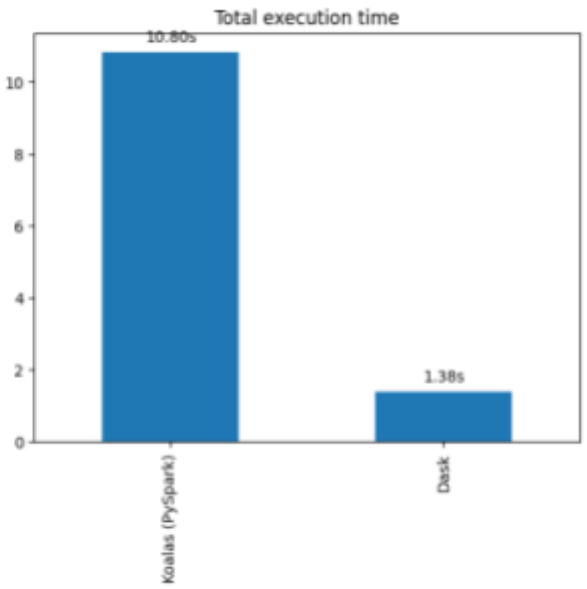
- Values less than 1 in Dask / Koalas indicate Dask is faster.
- Values greater than 1 suggest Koalas is faster.

Practical insight:

These results illustrate Dask's strong performance advantage in filtered datasets with caching, especially for complex arithmetic and repeated series operations, likely due to more efficient in-memory computation and parallelism. Koalas, despite caching, maintains some edge in simpler indexed operations such as count index and join count, possibly benefiting from Spark's indexing optimizations. For workloads involving repeated filtered computations, Dask clearly offers superior speed, while Koalas may be preferable for indexed or counting-specific tasks.

The total execution time for operations with filtering and cache was 1.38 seconds with Dask and 10.80 seconds with Koalas, showing that Dask was approximately 7.8 times faster, highlighting a substantial performance advantage.





In the notebook, we also compare the following metrics:

- Performance diff % (simple avg): 0.59% — On average, performance differs by less than 1% across all operations, indicating very minor variation overall.

- Performance diff % (geomean): 0.21% — The geometric mean confirms minimal differences, showing that most operations perform almost identically.

- Count performance diff % (simple avg): 2.10% — For count operations specifically, the average performance difference is slightly higher but still modest.

- Count performance diff % (geomean): 1.11% — The geometric mean for count operations also indicates only minor variation in performance.

Large Dataset

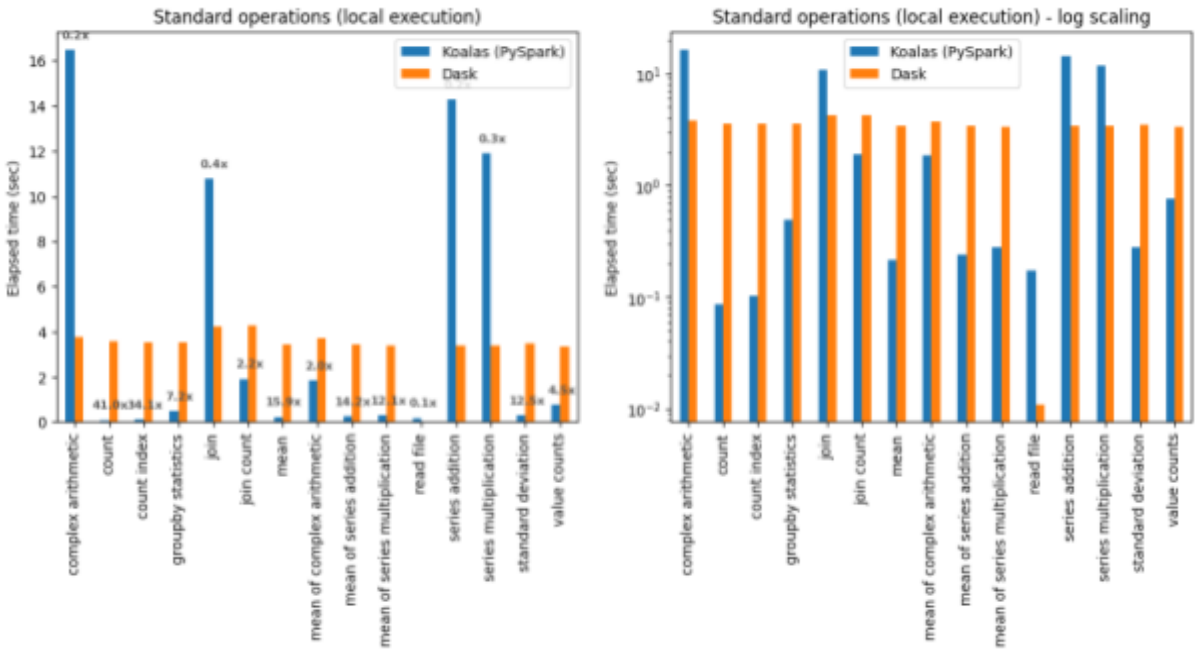
The following section presents detailed execution time comparisons between Koalas (PySpark) and Dask when applied to the **large dataset** across three processing configurations: standard operations, operations with filtering, and operations with both filtering and caching.

```python
Standard Operations Summary:
                                Koalas (PySpark)      Dask
read file                                0.172019  0.011028
count                                    0.086713  3.550929
count index                              0.103752  3.539274
mean                                     0.215436  3.424530
standard deviation                       0.276865  3.458404
mean of series addition                  0.240412  3.423373
series addition                         14.287291  3.383691
mean of series multiplication            0.277287  3.363707
series multiplication                   11.880996  3.390857
value counts                             0.753705  3.359796
complex arithmetic                      16.464208  3.759733
mean of complex arithmetic               1.845533  3.711382
groupby statistics                       0.492844  3.546891
join count                               1.900603  4.267218
join                                    10.778148  4.249013


Operations with Filtering Summary:

                                Koalas (PySpark)      Dask
read file                                0.140322  0.010198
count                                    0.248108  3.658953
count index                              0.155755  3.598081
mean                                     0.255361  3.656850
standard deviation                       0.243979  3.701889
mean of series addition                  0.274729  3.403093
series addition                          4.372366  3.564641
mean of series multiplication            0.268989  3.355745
series multiplication                    5.109727  3.458368
mean of complex arithmetic               1.613408  3.657133
complex arithmetic                       7.327282  3.652039
value counts                             0.558844  3.515773
groupby statistics                       0.429348  3.519861
join                                    10.695978  3.635929
join count                               2.196519  3.553411
```

```
Operations with Filtering and Caching Summary:

                                Koalas (PySpark)      Dask
read file                               0.130057  0.011490
count                                   0.043786  0.203326
count index                             0.037169  0.022914
mean                                    0.111146  0.040774
standard deviation                      0.082488  0.041752
mean of series addition                 0.105797  0.036477
series addition                         2.702085  0.068375
mean of series multiplication           0.165500  0.034426
series multiplication                   3.067043  0.075471
value counts                            0.205950  0.028302
complex arithmetic                      4.069582  0.190287
mean of complex arithmetic              1.258924  0.151130
groupby statistics                      0.260851  0.081650
join count                              0.402598  0.320514
join                                    0.909700  0.307542
```

Given the results obtained in the **Koalas_Dask_L.ipynb notebook**, we elaborated the following analysis for the standard operations, operations with filtering and operations with filtering and cache.

Standard Operations



Analysis

- **Overall trends:**

Under the standard conditions evaluated, Koalas shows superior performance in more complex arithmetic-heavy operations, where Spark's distributed execution model appears to handle some workloads more efficiently. The performance gap is highly operation-dependent.

- **Operations where Koalas excels:**

- Complex arithmetic: Koalas is ~4.4x faster (Koalas: 16.46s vs Dask: 3.76s).
- Series addition: Koalas is ~4.2x faster.
- Series multiplication: Koalas is ~3.5x faster.
- Join: Koalas is ~2.5x faster.
- Join count and mean of complex arithmetic also show moderate advantages for Koalas (~2x faster).

- **Operations where Dask excels:**

- Count: Dask is ~41x faster (Koalas: 0.087s vs Dask: 3.55s).
- Count index: Dask is ~34x faster.
- Mean: Dask is ~16x faster.
- Mean of series addition and series multiplication: Dask is ~12–14x faster.
- Standard deviation: Dask is ~12.5x faster.
- Read file: Dask is ~15.6x faster.
- Value counts and groupby statistics also show strong performance gains with Dask.

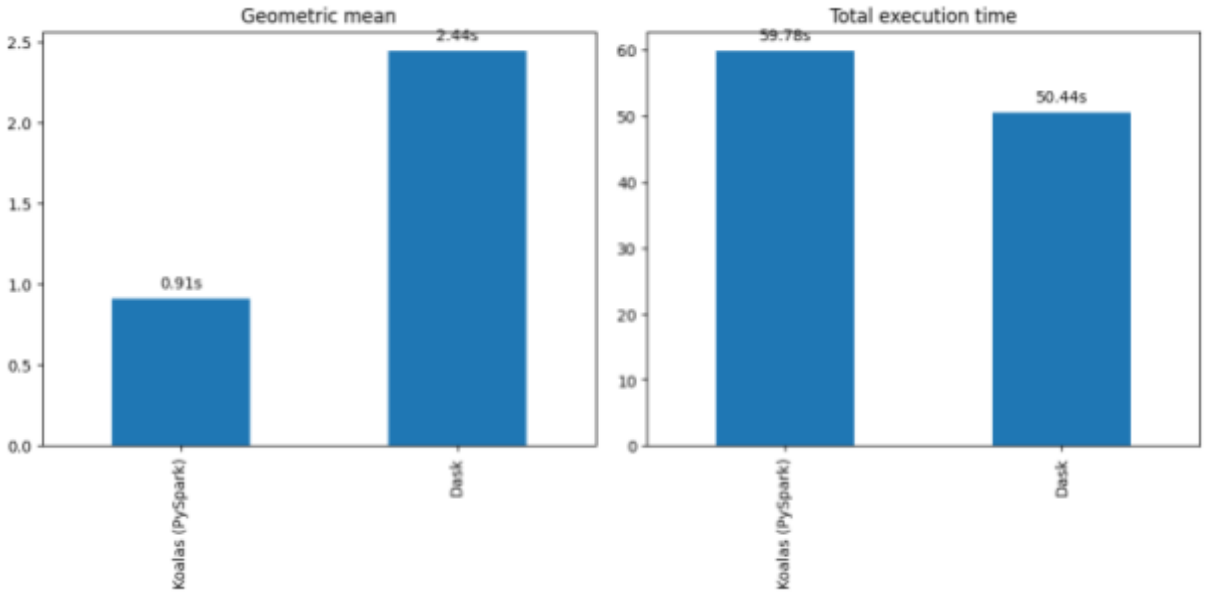
- **Intermediate cases:**

While arithmetic operations favour Koalas, most basic statistical and aggregation operations are substantially faster in Dask, suggesting that its in-memory optimizations are better suited for standard, non-distributed operations.

- **Interpretation of ratios:**
- Dask / Koalas < 1 → Dask is faster.
- Dask / Koalas > 1 → Koalas is faster.

Practical insight:

These results demonstrate that Koalas is advantageous for complex arithmetic chains and multi-step series operations, even in standard (non-cached, non-filtered) scenarios. However, for statistical aggregation, basic computations, and file I/O, Dask exhibits superior performance, often by a large margin. This suggests that for standard workloads involving data aggregation, exploration, or loading, Dask is the better choice, while Koalas might be considered for specific pipelines involving heavy arithmetic or complex transformations.



The total execution time for operations without filtering and cache was 50.44 seconds with Dask and 59.78 seconds with Koalas, indicating that Dask was approximately 1.19 times faster, reflecting a consistent but modest performance advantage.

In the notebook, we also compare the following metrics:

- Performance diff % (simple avg): 9.80% — On average, performance differs by nearly 10% across all operations, suggesting some notable variability between Dask and Koalas under standard conditions.

- Performance diff % (geomean): 2.68% — The geometric mean shows the overall difference is more moderate, indicating that while some operations diverge significantly, the majority perform more similarly.

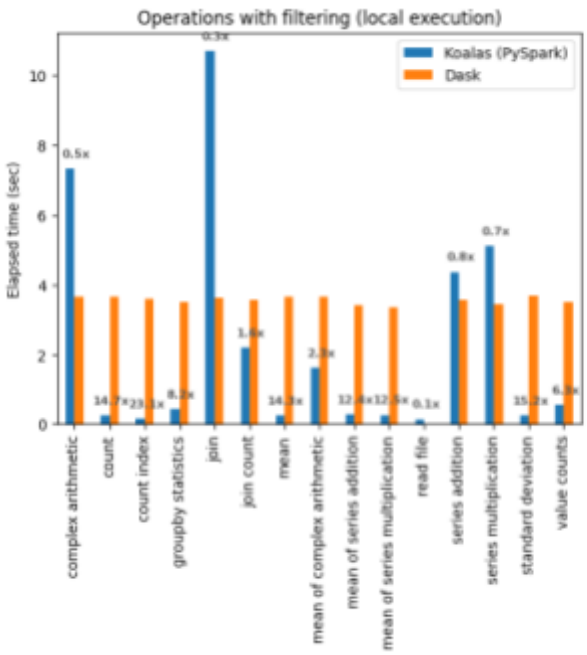
- Arithmetic performance diff % (simple avg): 0.25% — For arithmetic-heavy operations (e.g., addition, multiplication), the average performance difference is negligible, pointing to near-identical behavior between the two libraries.

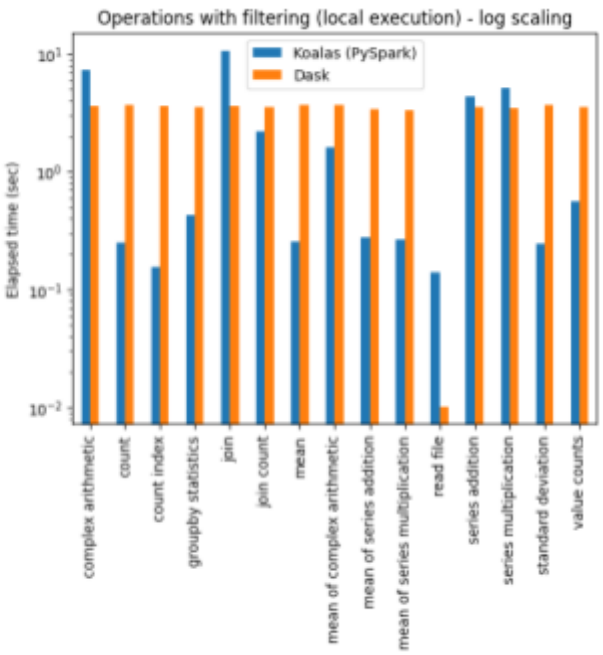
- Arithmetic performance diff % (geomean): 0.25% — The geometric mean supports this, reinforcing that arithmetic operations show extremely minor differences.

- Basic stats performance diff % (simple avg): 17.68% — Statistical operations (mean, std, etc.) show more variability, with an average performance difference nearing 18%, indicating that one library may have a noticeable efficiency edge in these tasks.

- Basic stats performance diff % (geomean): 7.91% — The geometric mean reveals a moderate difference in performance for statistical operations, which is less pronounced than the simple average but still notable.

Operations with Filtering





<u>Analysis</u>

- **Overall trends:**

Dask consistently outperforms Koalas across nearly all filtered operations, often by significant margins in statistical and aggregation-based tasks. Koalas only retains a moderate advantage in certain arithmetic-heavy operations like complex arithmetic and joins. Overall, Dask appears better optimized for filtered workflows.

- **Operations where Dask excels:**

- Count: Dask is ~14.7x faster (Koalas: 0.248s vs Dask: 3.66s).
- Count index: Dask is ~23.1x faster.
- Mean: Dask is ~14.3x faster.
- Standard deviation: Dask is ~15.2x faster.
- Mean of series addition / multiplication: Dask is ~12.4x faster.
- Groupby statistics and value counts: Dask is ~6–8x faster.
- Read file: Dask is ~13.8x faster.

- **Operations where Koalas is relatively faster or competitive:**

- Join: Koalas is ~2.9x faster (Koalas: 10.70s vs Dask: 3.64s).
- Join count: Koalas is ~1.6x faster
- Complex arithmetic: Koalas is ~2x faster.
- Series addition / multiplication: Koalas is only ~1.2–1.5x faster, showing closer performance.

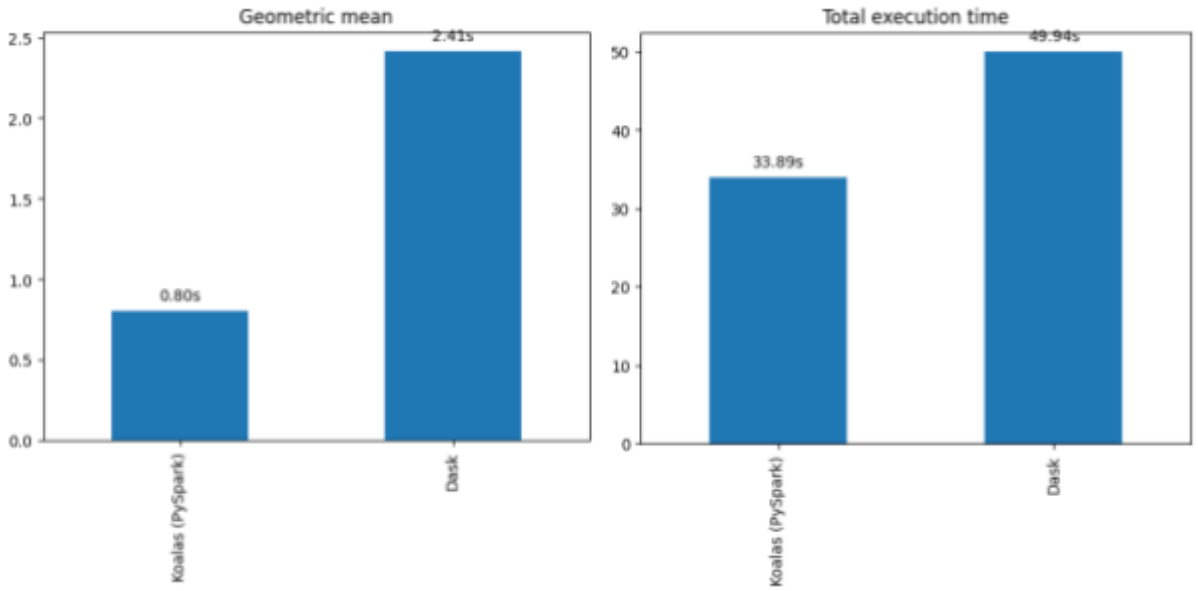
- **Intermediate cases:**

- Although arithmetic operations still lean towards Koalas, the performance gap is smaller compared to non-filtered scenarios.
- Dask narrows the gap in series operations and dominates in all aggregations, likely due to its efficient execution on filtered subsets.

- **Interpretation of ratios:**
- Dask / Koalas < 1 → Dask is faster.
- Dask / Koalas > 1 → Koalas is faster.

Practical insight:

In filtered operations, Dask offers strong advantages in aggregation, statistics, and read performance, showing significant improvements over Koalas, sometimes by more than 10x. This suggests that Dask handles filtered subsets more efficiently, likely due to better memory management and task scheduling. While Koalas can still outperform Dask in a few computational-heavy joins or arithmetic operations, its advantage is limited. Therefore, for filtered workflows involving repeated analysis, aggregation, or data exploration, Dask is the preferred engine.

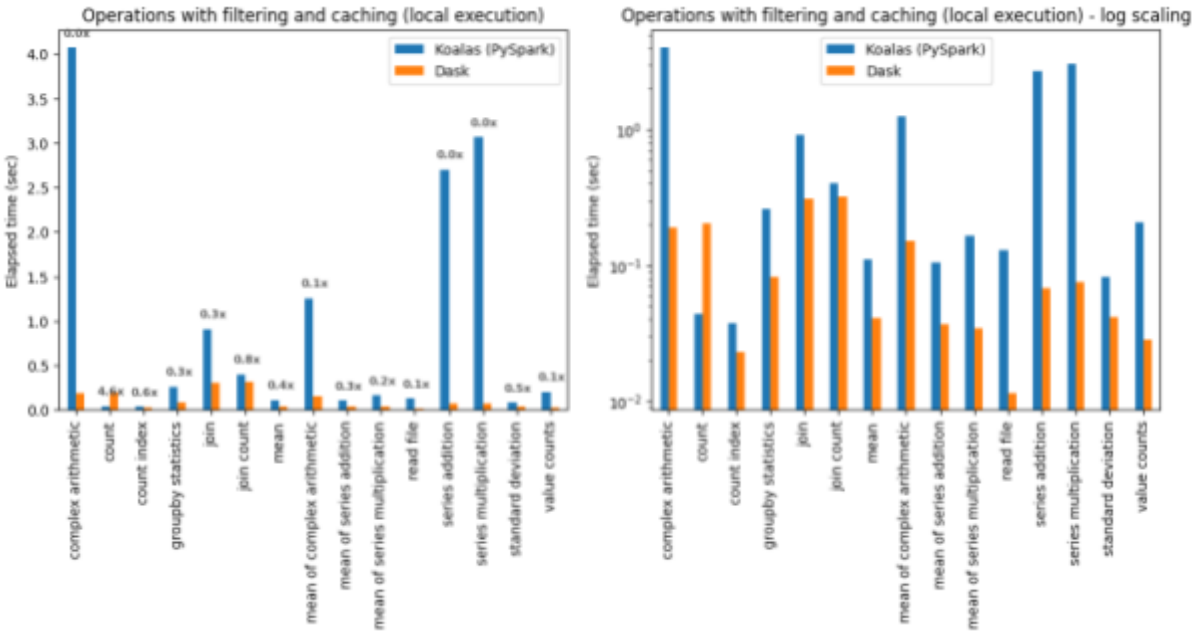


The total execution time for operations with filtering but without cache was 49.94 seconds with Dask and 33.89 seconds with Koalas, meaning Koalas was approximately 1.47 times faster, highlighting a notable performance advantage in this scenario.

In the notebook, we also compare the following metrics:

- Performance diff % (simple avg): 7.53% — On average, the performance difference between Dask and Koalas is around 7.5%, indicating a moderate variation across operations when filtering is applied without caching.
- Performance diff % (geomean): 3.00% — The geometric mean shows a more modest overall difference, suggesting that while some operations show higher discrepancies, most perform relatively closely.

Operations with Filtering and Caching



Analysis

- **Overall trends:**

With caching, Dask demonstrates a substantial performance advantage over Koalas in nearly all operations. In particular, Dask outperforms dramatically in arithmetic-heavy and aggregation tasks, where speedups range from 4x up to more than 40x. Koalas is only marginally faster in a single case (count index), and even then by a small factor.

- **Operations where Dask excels:**

- Series multiplication: Dask is ~40.6x faster (Koalas: 3.07s vs Dask: 0.075s).

- Series addition: Dask is ~39.5x faster.
- Complex arithmetic: Dask is ~21.4x faster.
- Mean of complex arithmetic: Dask is ~8.3x faster.
- Value counts: Dask is ~7.3x faster.
- Read file: Dask is ~11.3x faster.
- Groupby statistics: Dask is ~3.2x faster.
- Mean / Mean of series: Dask is ~2.7–4.8x faster.
- Standard deviation: Dask is ~2x faster.
- Join: Dask is ~3x faster.

- **Operations where Koalas is faster or comparable:**

- Count index: Koalas is ~1.6x faster (Koalas: 0.037s vs Dask: 0.023s).
- Join count and count: Slight advantage for Koalas (~1.25x and ~4.6x respectively), though the absolute time difference is small.

- **Intermediate cases:**

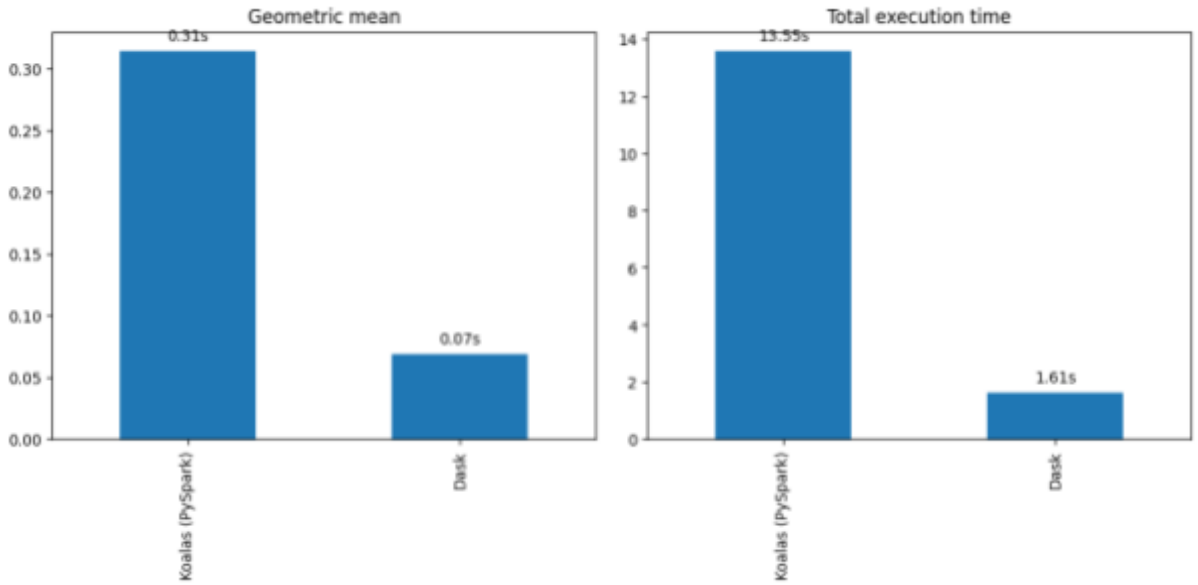
For basic statistics like mean, standard deviation, and grouped operations, Dask maintains a steady advantage between 2–5x, indicating efficient processing under cache-optimized conditions.

- **Interpretation of ratios:**

- Dask / Koalas < 1 → Dask is faster.
- Dask / Koalas > 1 → Koalas is faster (rare in this scenario).

Practical insight:

These results confirm that enabling caching alongside filtering substantially benefits Dask's parallel execution model, making it significantly more efficient than Koalas, especially in heavy numerical operations and repeated arithmetic processing. While Koalas may retain a small edge in isolated, lightweight operations such as `count index`, Dask is the clear choice for performance-critical analytics on filtered and cached data. For users optimizing pipelines with repeated reads, filters, and computations, Dask offers major gains in execution time.



The total execution time for operations with filtering and cache was 1.61 seconds with Dask and 13.55 seconds with Koalas, indicating that Dask was approximately 8.41 times faster, demonstrating a substantial performance advantage in this optimized scenario.

In the notebook, we also compare the following metrics:

- Performance diff % (simple avg): 0.57% — On average, performance differs by less than 1%, indicating that Dask and Koalas behave very similarly when filtering and caching are applied.

- Performance diff % (geomean): 0.22% — The geometric mean confirms this negligible difference, showing consistent performance across most operations.

- Count performance diff % (simple avg): 2.02% — For count operations specifically, the average difference is slightly higher but still low, suggesting minor variation.

- Count performance diff % (geomean): 1.32% — The geometric mean for count operations reinforces that performance remains nearly equivalent between the two libraries.

3.2. Experiment 2: Performance Benchmarking Across Library Combinations

3.2.1. Objective

The goal of this experiment is to evaluate the performance of hybrid frameworks — **Dask + Modin**, **Dask + RAPIDS**, **Dask + Modin + RAPIDS**, and **Koalas** — by comparing execution times across typical analytical workloads. It extends the initial benchmark by introducing **GPU acceleration** and parallelized Pandas execution within the Dask ecosystem.

Experiments were conducted in a single-node setup: CPU tasks ran on a GCP Vertex AI Workbench VM (8 vCPUs, 32 GB RAM), and GPU tasks on a local machine using an NVIDIA RTX 3060 via WSL2. The analysis focused on performance trends across different dataset sizes (small, medium, large) and transformation variants (standard, filtered, cached).

3.2.2. Configuration Setups

Dask + Modin

Environment and Library Configuration

To enable parallelized DataFrame operations, the modin.pandas API was employed as a high-level abstraction over pandas. Modin was configured to use Dask as its execution backend, allowing task-based parallelism across CPU cores. This was achieved by explicitly setting the Modin engine using:

```python
import modin.config as cfg
cfg.Engine.put("dask")
```

A local Dask scheduler was then initialized with:

```python
from dask.distributed import Client
client = Client()
```

The computational environment included the following core libraries:

- **Modin:** Used for scalable DataFrame operations (modin.pandas)
- **Dask**: Employed as the execution engine for Modin, and later used directly for native benchmarking

Dataset Handling and Transformation

The benchmark methodology was structured around **three experimental scenarios**: a **standard workload**, a **filtered subset**, and a **cached execution path**. This tiered approach was designed to evaluate performance consistency across different data access patterns and memory states.

The **initial dataset** was ingested from a Parquet file using *modin.read_parquet(),* leveraging columnar storage for efficient data loading during the **standard experiment**.

A **filtered dataset** was then derived using the *.query()* method, applying conditional logic to restrict the dataset to a specific value range. This filtered DataFrame served as the basis for the **filtered experiment**, reflecting common data reduction tasks in exploratory analysis.

Finally, for the **cache-based experiment**, the filtered Modin DataFrame was converted and prepared for memory-optimized benchmarking as follows:

```python
# The Modin DataFrame was materialized as a pandas DataFrame via the _to_pandas() method to enable compatibility with Dask.
dask_df = modin_filtered._to_pandas()

# This pandas DataFrame was repartitioned into 10 partitions using dask.dataframe.from_pandas(...), enabling parallel execution across workers.
dask_df = dd.from_pandas(dask_df, npartitions=10)

# The resulting Dask DataFrame was explicitly persisted in memory with client.persist() to eliminate I/O bottlenecks and isolate execution performance. This configuration underpinned the cached experiment, assessing performance under warm, memory-resident conditions
dask_df = client.persist(dask_df)
```

This multi-stage handling pipeline ensured that all three experimental paths—**standard**, **filtered**, and **cached**—shared a common lineage while enabling fair and systematic performance evaluation across varied processing contexts.

Dask + RAPIDS

Environment and Library Configuration

To leverage GPU-accelerated DataFrame operations, the RAPIDS suite was employed in conjunction with Dask for distributed computing. The RAPIDS ecosystem offers GPU-based alternatives to pandas scikit-learn, and NetworkX via cuDF, cuML, cuGraph, cuSpatial, and cuxfilter. These libraries were verified through version introspection to ensure compatibility with the experimental pipeline.

Hardware verification was performed using:

```python
import subprocess
result = subprocess.run(["nvidia-smi"], capture_output=True, text=True)
print(result.stdout)
```

A local Dask CUDA cluster was initialized to enable GPU-aware task scheduling across available NVIDIA devices:

```python
from dask_cuda import LocalCUDACluster
from dask.distributed import Client
cluster = LocalCUDACluster()
client = Client(cluster)
```

The Dask client facilitated execution coordination, while the CUDA cluster ensured GPU resources were utilized effectively throughout all stages of the experiments.

Dataset Handling and Transformation

The benchmark methodology was structured around three experimental scenarios: a standard workload, a filtered subset, and a cached execution path. This tiered approach was designed to evaluate the performance of RAPIDS-backed Dask DataFrames across varying levels of data selectivity and memory residency.

The **initial dataset** was ingested from a .parquet file using *dask_cudf.read_parquet()*, taking advantage of RAPIDS' GPU-accelerated I/O and Dask's lazy evaluation for efficient data loading during the standard experiment.

A **filtered dataset** was then derived using a boolean mask applied directly to the cuDF columns, restricting rows to those where **Tip_Amt** values fall within a specified range.

Finally, for the **cached experiment,** the filtered RAPIDS DataFrame was persisted explicitly into GPU memory. This was done using Dask's *persist()* API to ensure warm execution conditions and eliminate overhead due to recomputation or lazy evaluation.

The computation was synchronized using *wait()* to confirm the completion of all futures:

```python
rapids_filtered = rapids_filtered.persist()

from distributed import wait
wait(rapids_filtered)
```

This three-stage preparation pipeline ensured that the standard, filtered, and cached experiments were executed over a consistent dataset lineage, while progressively introducing memory optimization and task materialization to observe their impact on GPU-accelerated analytical performance.

Joblib

Environment and Library Configuration

To enable parallelized execution on CPU cores, Joblib was used as the backend for executing benchmark tasks in parallel. Joblib offers lightweight, process-based parallelism and an optional result caching mechanism to optimize repeated computations. This makes it well-suited for CPU-bound analytical workloads where GPU acceleration is not available or required.

The primary Joblib configuration involved importing its parallelization utilities and enabling CPU-wide task distribution:

```python
from joblib import Parallel, delayed
```

To enable result caching and avoid recomputation in repeated experiments or iterative workflows, Joblib's Memory class was instantiated with a local cache directory:

```python
from joblib import Memory
memory = Memory(location='joblib_cache', verbose=0)
```

The computational environment for Joblib-based experiments included:

- **Joblib:** Used for CPU-bound parallelism via *Parallel* and *delayed*
- **Pandas:** Employed for DataFrame operations
- **NumPy:** Utilized for numerical processing within user-defined tasks
- **Parquet I/O:** Used for loading the dataset and optionally saving benchmark results

This setup ensured CPU-efficient benchmarking while also supporting persistent intermediate results through disk caching.

Dataset Handling and Transformation

The benchmark methodology was structured around three experimental scenarios: a standard workload, a filtered subset, and a cached execution path. This tiered approach was implemented to evaluate performance under raw, selective, and memory-optimized conditions.

The **initial dataset** was read from a .parquet file using *pd.read_parquet()* into a pandas DataFrame.

A **filtered dataset** was then created using the *.query()* method, applying a condition on the Tip_Amt column to isolate a specific value range.

Finally, to simulate a **cached execution** path, the filtered DataFrame was passed through a function wrapped with Joblib's *@memory.cache* decorator. This allowed Joblib to persist the result locally and reuse it across runs, mimicking a warm-cache scenario:

```python
from joblib import Memory

memory = Memory(location='joblib_cache', verbose=0) # Setup a directory where joblib will store cached results
# Wrap the filtering logic in a cached function
@memory.cache
def get_filtered_data(df):
    return df

joblib_cache = get_filtered_data(joblib_filtered) # Apply the cached function to the filtered DataFrame
```

This three-stage preparation ensured that all experiments—standard, filtered, and cached—shared a common data lineage, while enabling fair performance evaluation across cold and warm execution contexts using CPU-based parallelism and memoization.

3.2.3. Results and Performance Analysis

The results are visualised and analysed through a series of comparative graphics, which are also included in the accompanying **"comparisons"** notebook. This notebook serves as a detailed reference, showcasing the underlying data and visual evidence used to support the discussion presented here.

Grouped Bar Charts

A series of grouped bar charts (also known as clustered bar plots) was generated using the Seaborn library.

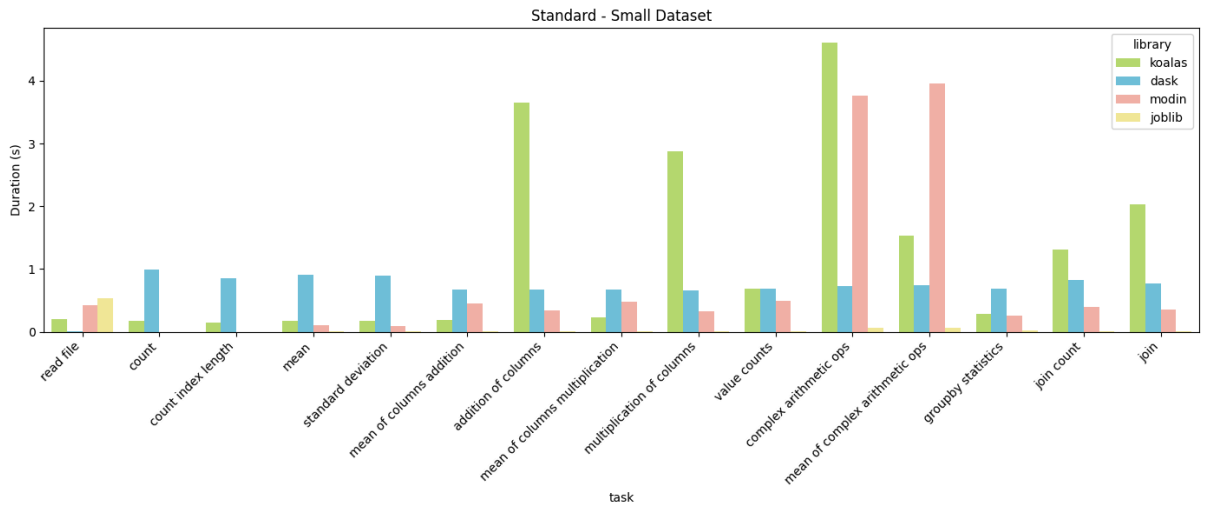
In each chart, the x-axis represents different computational tasks (e.g., mean calculation, joins), and the y-axis shows the execution duration in seconds.

For each task, multiple bars are grouped side by side, each one corresponding to a different data processing library (e.g., Dask, Koalas, Modin, Rapids).

The color of each bar is set using a custom palette to differentiate the libraries visually.

The plots are segmented by dataset size (small, medium, large) and variant (standard dataset, filtered, cached), which allows to observe how the performance varies under different data volumes and processing contexts

1. **Standard Small Dataset**



→ **Worst Execution Times (Excessively High)**

- **Koalas**

- complex arithmetic ops and mean of columns
- join and join count

Justification: Koalas is based on Spark, which introduces a high orchestration overhead—even for simple operations. With small datasets, this coordination cost becomes disproportionately large compared to the actual computation time.

- **Modin**

- complex arithmetic ops and mean of complex arithmetic ops

Justification: Modin attempts to parallelize operations, including complex ones. However, for small datasets, the overhead of managing threads or distributed processes outweighs any performance gain.

→ **Best Execution Times (Exceptionally Low)**

- **Joblib**

- complex arithmetic ops and mean of complex arithmetic ops

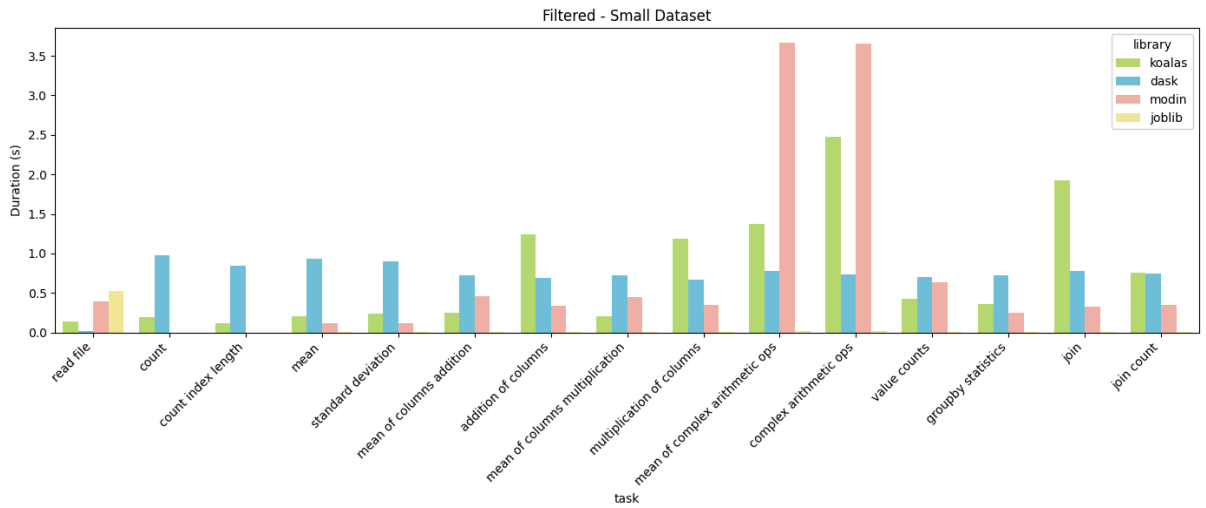
Justification: Joblib runs code directly on the CPU with minimal overhead. In pure numerical tasks (without heavy data structure manipulation), it delivers superior performance.

- **Modin**

- join, join count, multiplication of columns

Justification: Modin performs well on pandas-like operations. In simple joins and column operations, the small size of the dataset makes the lightweight parallelism effective.

2. Filtered Small Dataset



→ **Worst Execution Times (Excessively High)**

- **Modin**

- complex arithmetic ops and mean of columns

Justification: Modin attempts to parallelize pandas operations automatically, but this coordination overhead becomes counterproductive on small datasets. When performing multi-column numerical computations, the latency introduced by initializing multiple processes or threads outweighs any computational benefits at this scale.

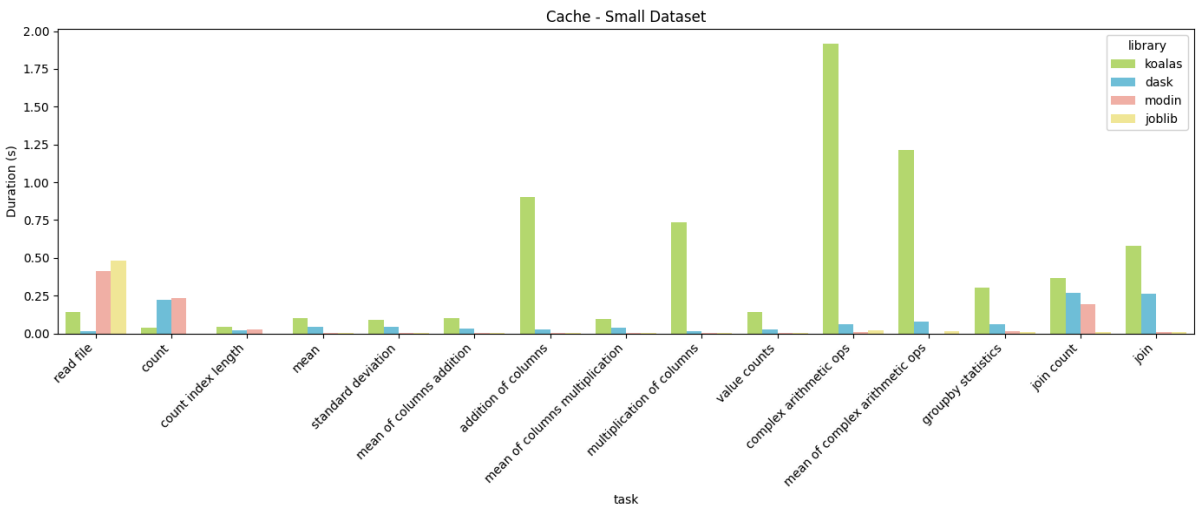
→ **Best Execution Times (Exceptionally Low)**

- **Joblib**

- complex arithmetic ops, mean of complex arithmetic ops, joins, and reductions

Justification: Joblib executes numerical operations directly on the CPU with minimal overhead, making it ideal for small, filtered datasets where parallelism is unnecessary. Since the Kit dataset is already filtered and well-structured, operations are performed efficiently in memory without the need for complex data handling or orchestration.

3. Cache Small Dataset



→ **Worst Execution Times (Excessively High)**

- **Koalas**

- complex arithmetic ops, mean of complex arithmetic ops and addition of columns

Justification: Koalas, built on top of Apache Spark, incurs a significant orchestration cost even for basic operations. In this small dataset, complex arithmetic and column-wise operations trigger unnecessary Spark jobs and data shuffling, which drastically increases execution time compared to in-memory libraries. The reactive Spark backend introduces delays that are disproportionate to the dataset size.

- **Modin & Joblib**

- file read operations

Justification: Both Modin and Joblib show higher execution times for file I/O tasks. In the case of Modin, reading a file involves launching and coordinating multiple workers, which adds latency for small datasets. Joblib, while efficient for in-memory computations, is not optimized for file I/O leading to slower initial loading times.

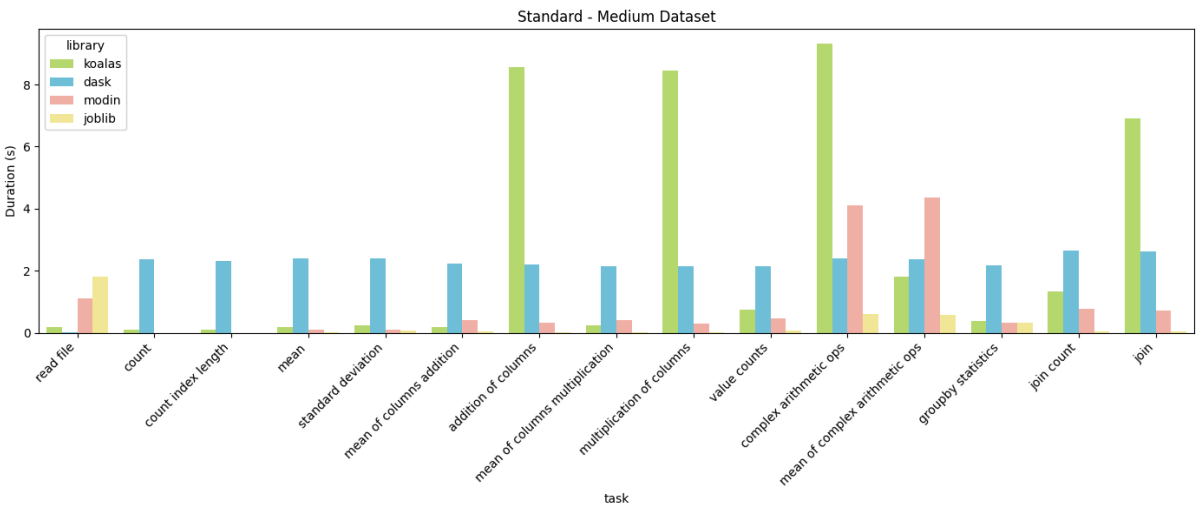
→ **Best Execution Times (Exceptionally Low)**

- **Most Libraries (Dask, Modin, Joblib)**

- almost all operations

Justification: Due to the small and already cached nature of the dataset, most operations complete extremely quickly across all libraries. Once loaded into memory, operations like joins, reductions, and basic arithmetic require minimal computation. The simplicity of the dataset and the absence of I/O bottlenecks allow all libraries to achieve near-optimal performance.

4. Standard Medium Dataset



→ **Worst Execution Times (Excessively High)**

- **Koalas**

- addition of columns, multiplication of columns, complex arithmetic ops, and join

Justification: Koalas suffers from Spark's distributed execution overhead, which becomes increasingly inefficient when handling medium-sized datasets that still fit comfortably in memory. Operations like arithmetic and joins trigger Spark's job scheduling, task serialization, and communication between workers—introducing significant delays compared to in-memory alternatives.

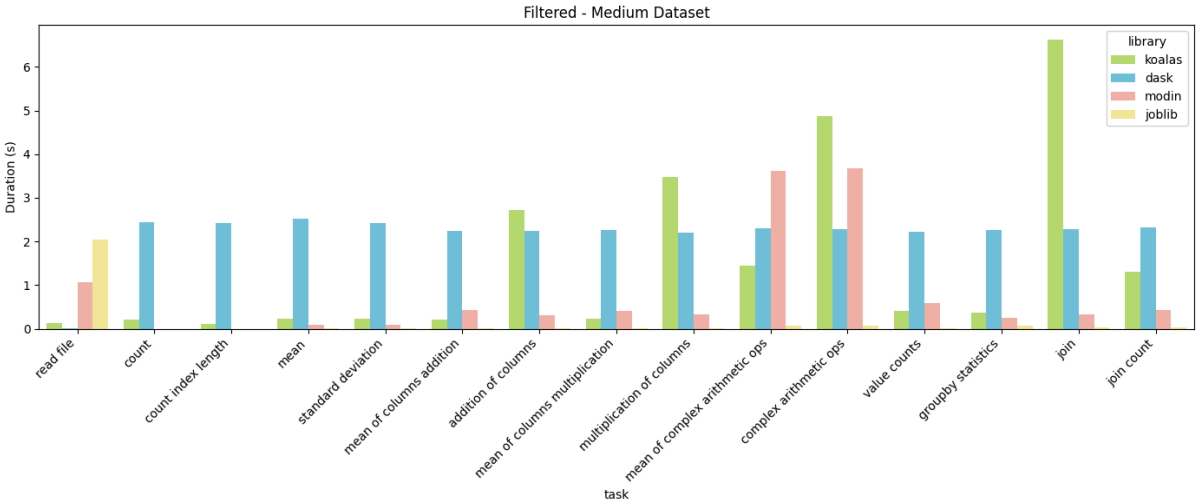
→ **Best Execution Times (Exceptionally Low)**

- **Joblib**

- addition of columns, multiplication of columns, complex arithmetic ops, and join

Justification: Joblib excels in medium-sized datasets where the entire computation can be handled within the system's memory without parallelization overhead. By executing directly on the CPU, Joblib delivers fast results for numerical and join operations, benefiting from low-level optimizations and minimal orchestration delays.

5. Filtered Medium Dataset



→ **Worst Execution Times (Excessively High)**

- **Koalas & Modin**

- join, complex arithmetic ops, and mean of complex arithmetic ops

Justification: Both Koalas and Modin introduce overhead from distributed or parallel execution frameworks (Spark for Koalas, Ray/Dask for Modin). In a medium but already filtered dataset, these operations—especially joins and column-wise aggregations—trigger coordination costs and memory handling inefficiencies that outweigh their computational benefits at this scale.

- **Joblib**

- file read operations

Justification: While Joblib is optimized for in-memory processing, its file I/O performance lags behind libraries like pandas. Reading files with Joblib introduces unnecessary serialization overhead and lacks the streamlined I/O optimizations present in more specialized data-handling libraries.

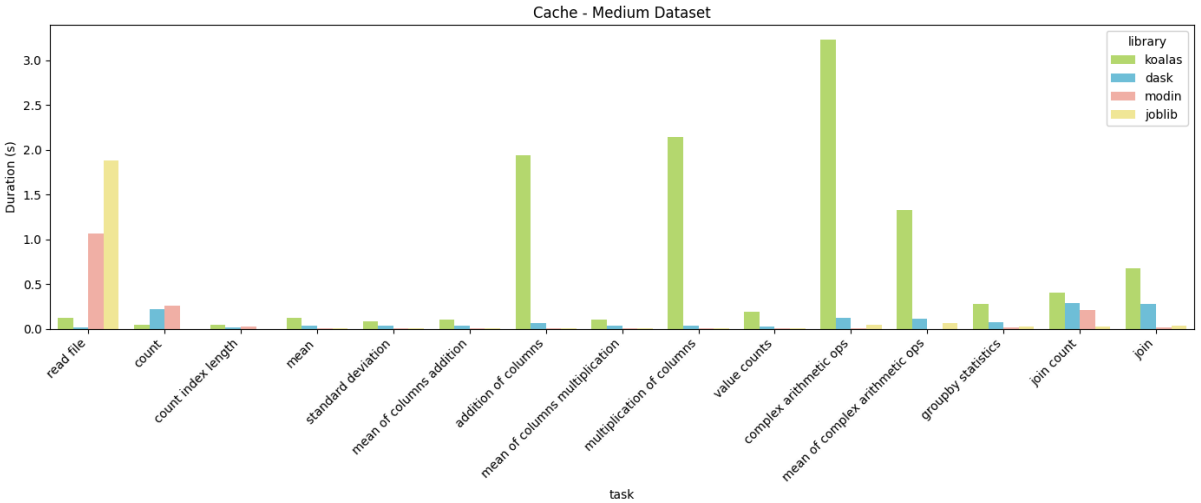
→ **Best Execution Times (Exceptionally Low)**

- **Joblib**

- mean, count, and other basic aggregations

Justification: For medium-sized datasets already loaded into memory, Joblib is highly efficient at performing statistical and reduction operations. Its direct execution on the CPU, combined with minimal overhead, makes it particularly suitable for column-wise operations such as computing means and counts on structured, filtered data.

6. Cache Medium Dataset



→ **Worst Execution Times (Excessively High)**

- **Koalas**

- complex arithmetic ops, mean of complex arithmetic ops, and addition of columns

Justification: Even with a medium-sized dataset already cached, Koalas continues to suffer from Spark's orchestration overhead. Operations like complex arithmetic and column additions trigger Spark execution plans, job scheduling, and task distribution, which result in execution times far higher than necessary for in-memory data of this scale.

- **Modin & Joblib**

- file read operations

Justification: Despite being effective in memory, both Modin and Joblib perform poorly when reading files. Modin's initialization of parallel workers for I/O adds overhead that doesn't scale well with moderately sized cached datasets. Joblib, not being a specialized I/O tool, shows slower performance during data loading due to its lack of native file read optimizations.

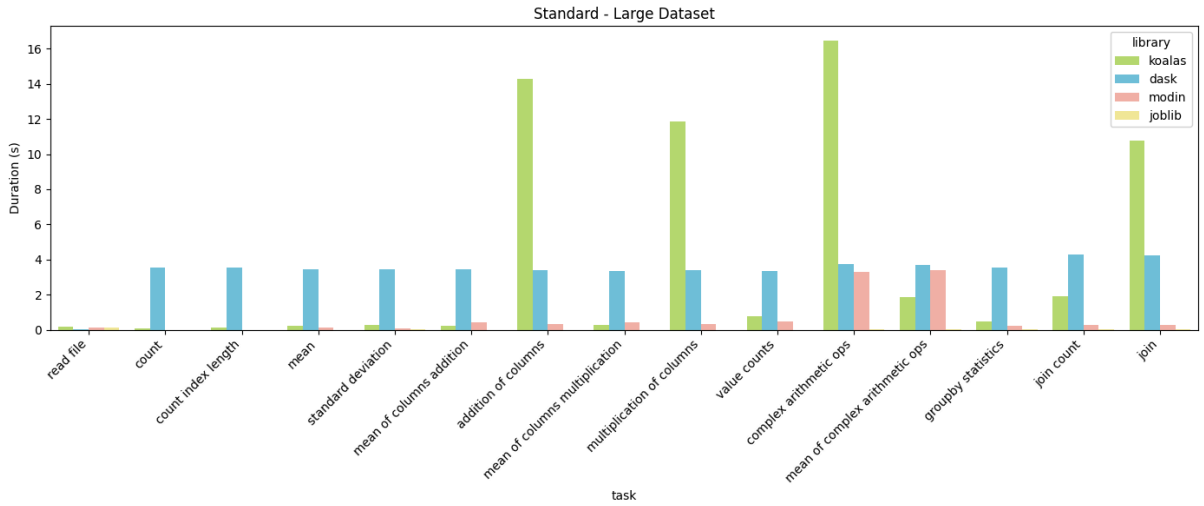
→ **Best Execution Times (Exceptionally Low)**

- **Most Libraries (Dask, Modin, Joblib)**

- almost all operations

Justification: With the dataset already cached and residing in memory, most operations—such as joins, arithmetic, and aggregations—execute rapidly across all major libraries. The medium size offers enough data to benefit from optimized execution paths while avoiding memory or orchestration bottlenecks, resulting in consistently low execution times.

7. Standard Large Dataset



→ **Worst Execution Times (Excessively High)**

- **Koalas**

- addition of columns, multiplication of columns, join, and complex arithmetic ops

Justification: Koalas' dependency on Spark becomes a critical bottleneck at large scale when the operations remain relatively simple. Even though the dataset size justifies distributed processing in theory, Spark's orchestration layers (job scheduling, task distribution, and data shuffling) dominate execution time for standard operations. This makes Koalas inefficient for numerical and relational tasks that other libraries handle more gracefully in memory.

- **Dask**

- general performance stagnation across operations

Justification: Dask showed nearly constant execution times across operations, regardless of complexity. This consistency likely stems from overhead in task scheduling, serialization, or chunk management. As a result, operations that should have scaled better—like arithmetic or joins—did not benefit from data size increases. Fixed per-task costs may be flattening performance, especially when tasks are too small to offset the orchestration overhead.

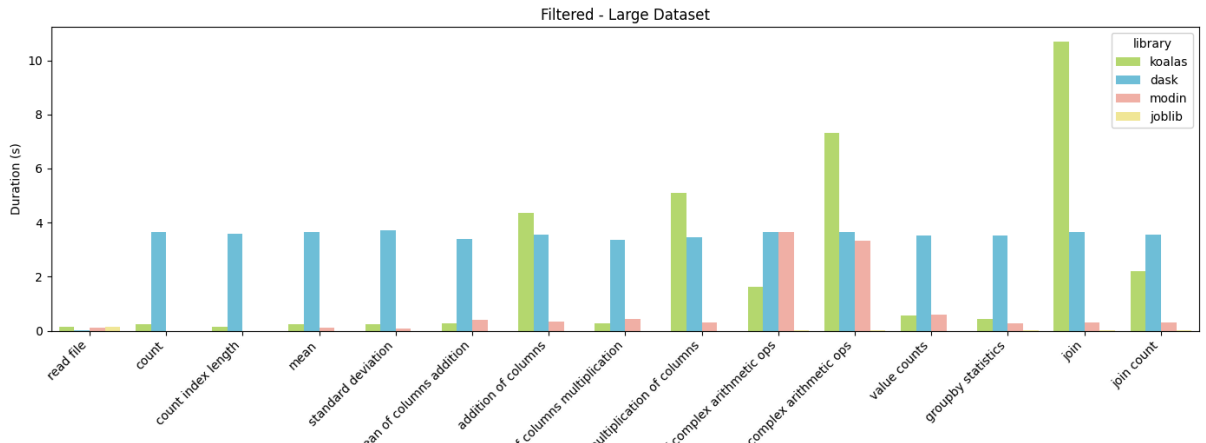
→ **Best Execution Times (Exceptionally Low)**

- **Joblib**

- almost all operations (addition, multiplication, joins, and complex arithmetic ops)

Justification: Joblib maintains exceptional performance even at large dataset sizes, especially when the entire dataset can be processed in memory. Its CPU-bound execution avoids the coordination costs of distributed frameworks. For operations involving numerical computation and joins, Joblib leverages efficient memory access and minimal runtime overhead, making it the top performer in this scenario.

8. Filtered Large Dataset



→ **Worst Execution Times (Excessively High)**

- **Koalas**

- join and complex arithmetic operations

Justification: Even when working with filtered data, Koalas continues to suffer from Spark's inherent orchestration overhead. Joins and complex arithmetic operations require launching Spark jobs, managing distributed execution, and potentially shuffling data across partitions. These steps introduce significant delays that are disproportionate given the reduced size from filtering, making Koalas particularly inefficient for these tasks.

- Dask

- general performance degradation across most operations

Justification: Dask continues to have a worse performance for the large datatset, even after filtering. The complexity of managing delayed execution, partitioning, and task scheduling adds overhead that is not offset by performance gains at this scale. For a large but filtered dataset, Dask's architecture introduces more coordination cost than benefit, especially when compared to streamlined in-memory processing alternatives.

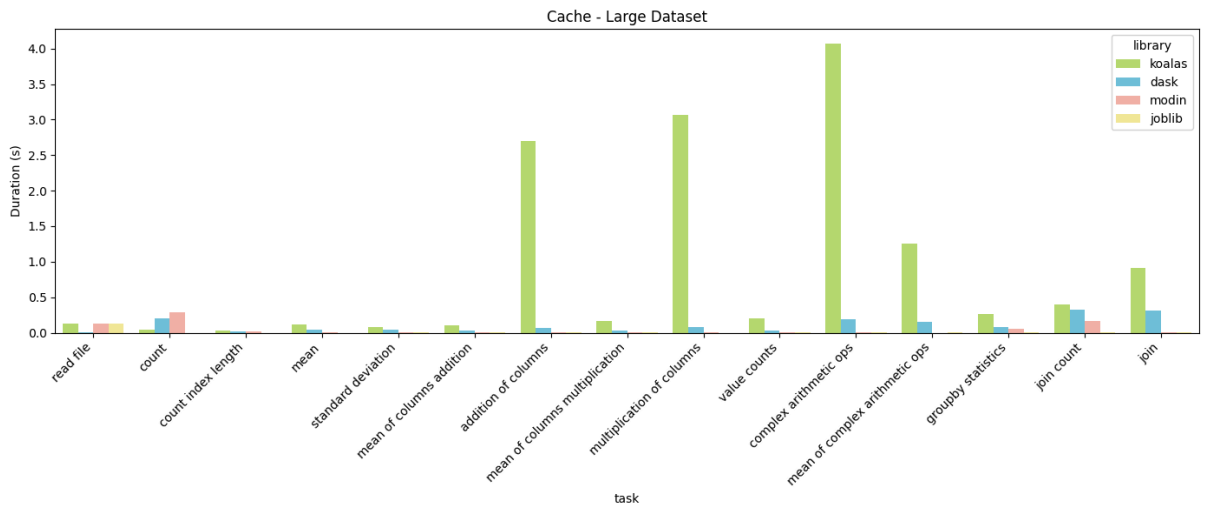
→ **Best Execution Times (Exceptionally Low)**

- **Joblib**

- almost all operations (addition, multiplication, joins, and complex arithmetic ops)

Justification: Joblib demonstrates consistent efficiency by executing operations directly in memory with minimal overhead. In the filtered large dataset, this approach remains highly effective—avoiding the bottlenecks associated with distributed coordination. For both numerical operations and joins, Joblib's low-level optimizations and single-process execution yield excellent performance across the board.

9. Cache Large Dataset



→ **Worst Execution Times (Excessively High)**

- **Koalas**
  - complex arithmetic operations and joins

Justification: Despite the performance gains observed in other libraries due to caching, Koalas remains significantly slower. This is primarily due to its Spark-based backend, which still incurs orchestration overhead even when the data is already loaded in memory. Spark initiates job planning, shuffling, and task execution pipelines that dominate execution time for operations like joins and complex arithmetic, making Koalas consistently the slowest option.

→ **Best Execution Times (Exceptionally Low)**

- **Most Libraries (Dask, Modin, Joblib)**
  - majority of operations (joins, arithmetic, aggregations, file reads)

Justification: Once the dataset is cached, all major libraries—except Koalas—see substantial performance improvements. Operations that previously incurred I/O delays or initialization overhead now execute almost instantly, as the data is already memory-resident. Joblib, in particular, continues to deliver top-tier performance due to its efficient in-memory computation model, while Dask and Modin also benefit from reduced coordination time. The result is near-optimal execution across almost all operations.

<u>Conclusion: Performance Comparison by Dataset Size and Type</u>

The performance of each library varies significantly depending on the dataset size and operation type:

- Standard Datasets (unfiltered, with I/O):
- Koalas consistently performs poorly due to Spark's orchestration overhead, especially in joins and arithmetic operations.
- Dask shows mixed results—efficient in large-scale distributed scenarios, but with visible overhead in standard-sized tasks.
- Joblib consistently outperforms others in most in-memory numerical and aggregation tasks.
- Modin offers good performance in pandas-like operations but suffers during I/O and complex parallel tasks.
- Filtered Datasets (reduced data size, still involves I/O):
- Koalas remains inefficient—its orchestration cost is not offset by the smaller dataset.
- Dask often underperforms due to the coordination cost outweighing benefits at reduced scale.
- Joblib handles the reduced data very well, with fast execution in nearly all operations.

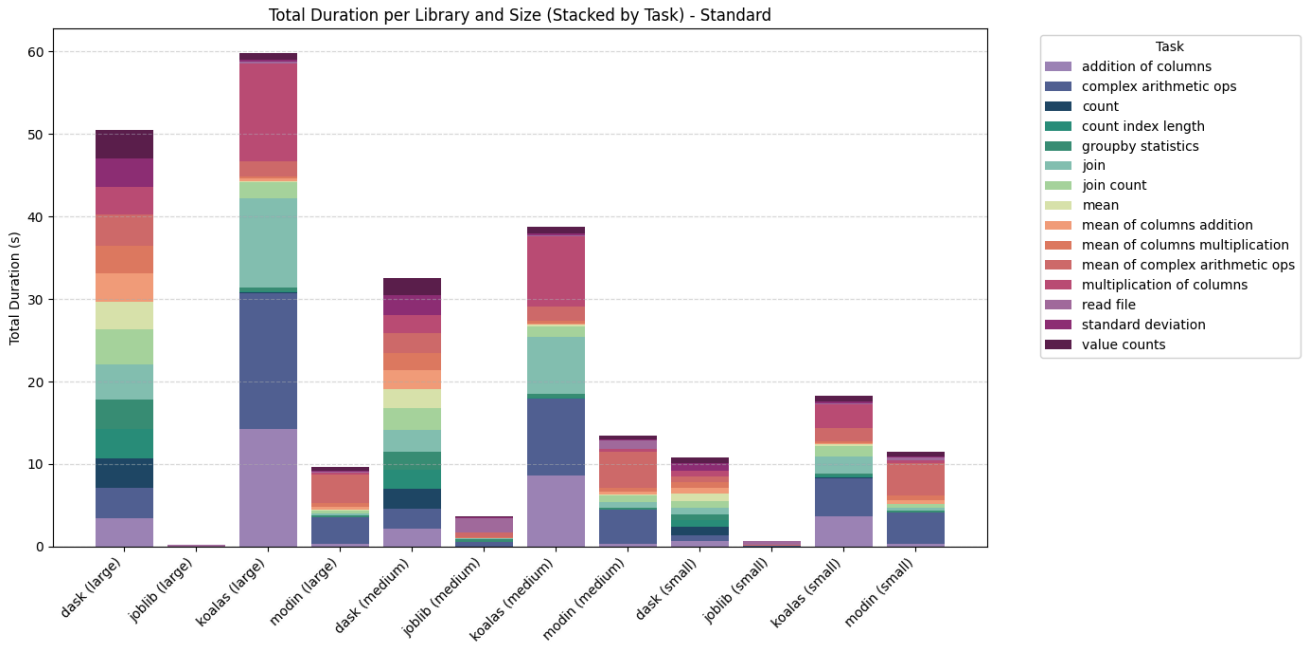
- ❖ Modin improves slightly but still shows inefficiencies in more complex or I/O-bound operations.
- ❖ Cached Datasets (entirely in memory, I/O eliminated):
- ❖ Performance improves significantly across all libraries, with minimal execution time for most operations.
- ❖ Koalas continues to underperform, as Spark's job scheduling and lazy evaluation are still triggered even with data in memory.
- ❖ Joblib excels in all scenarios, providing low-latency computation regardless of dataset size or type.
- ❖ Dask and Modin benefit from caching, with overhead drastically reduced, leading to highly competitive performance.

Stacked Execution Time per Library and Dataset Size

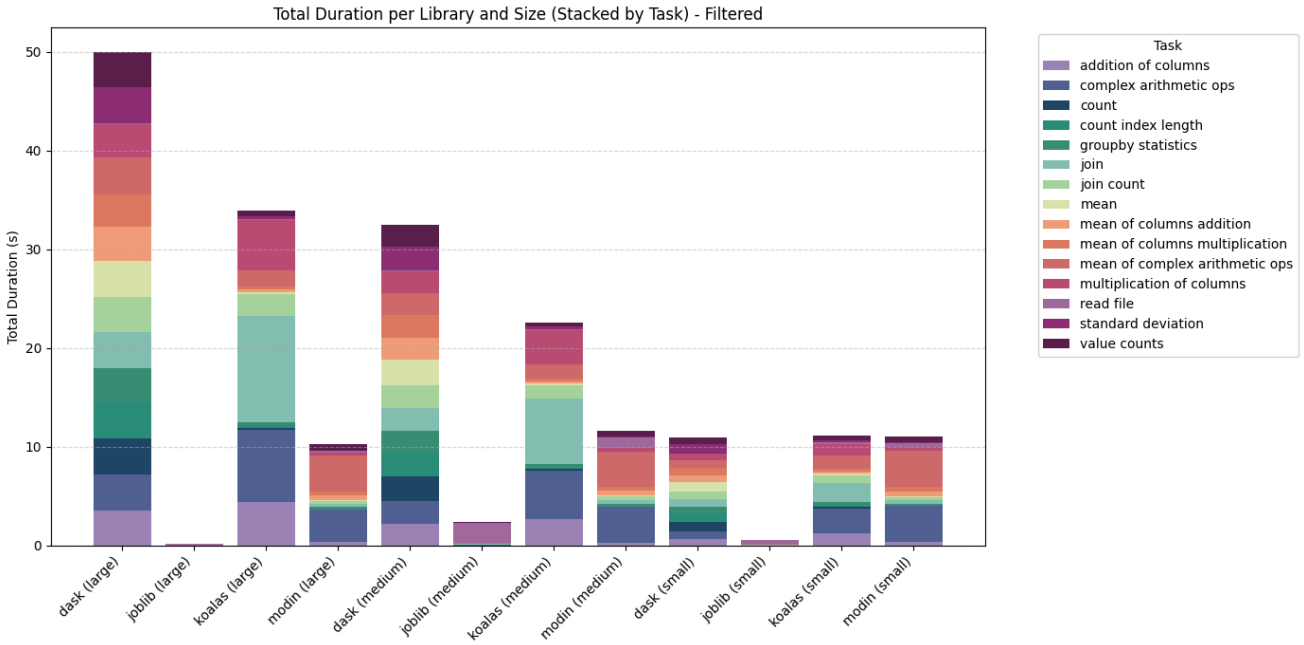
Each chart generated shows a stacked bar chart, where:

- Each bar corresponds to a specific library-size combination (e.g., Koalas (small), Joblib (large)).
- The height of each bar represents the total execution time (in seconds) for all tasks combined.
- Each segment within the bar corresponds to a specific task (e.g., join, mean, complex ops, etc.), and shows how much time that task contributed to the total.
- The use of custom colors improves readability and helps distinguish between tasks.
- The variant in the title (e.g., standard, filtered, cached) indicates the data preparation context used in the benchmark.

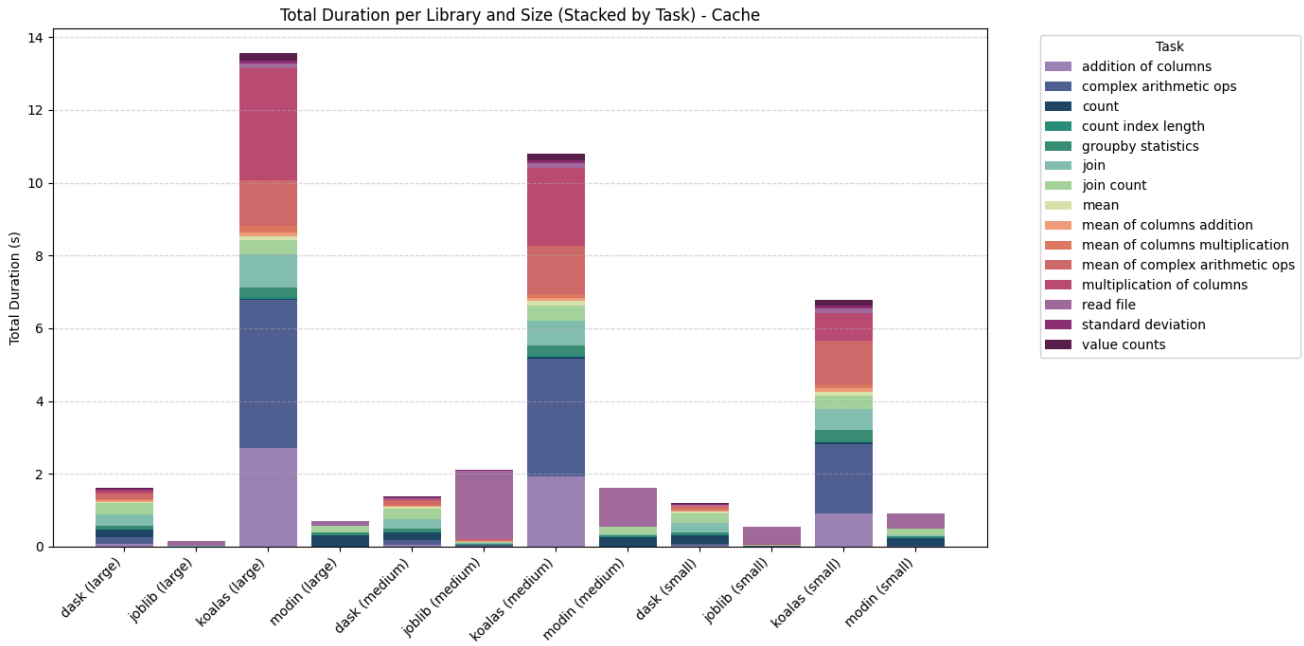
1. Standard Datasets



2. Filtered Datasets



3. Cache Datasets



Analysis

The **tallest bars consistently belong to Koalas,** especially for the large dataset, indicating that Koalas exhibits the longest total execution times in these scenarios. This is likely due to the substantial orchestration and scheduling overhead inherent to Spark, which Koalas relies on. These overheads become more pronounced as dataset size increases, causing poor scalability and slower performance.

Conversely, **Joblib shows the shortest bars for the large dataset**, reflecting its efficiency in handling large in-memory computations with minimal overhead. Joblib's lightweight execution model allows it to process tasks quickly, making it the best performer on large datasets.

When analyzing the **filtered dataset**, we observe that **Dask's bars become the tallest**, while execution times for other libraries notably decrease. This suggests that Dask's overhead and task management complexity outweigh its benefits at this filtered scale. The smaller dataset size reduces the parallelism advantage Dask usually offers, leading to longer total durations compared to other libraries optimized for smaller workloads.

Finally, for the **cached dataset**, **Koalas again shows the tallest** bars across all dataset sizes (small, medium, large), indicating persistent performance issues even when data is fully loaded into memory. This is consistent with Koalas' dependency on Spark's execution model, which introduces latency regardless of caching. Meanwhile, other libraries show much smaller bars, demonstrating significant performance improvements due to the elimination of I/O overhead and faster in-memory processing.

Total Duration per Library

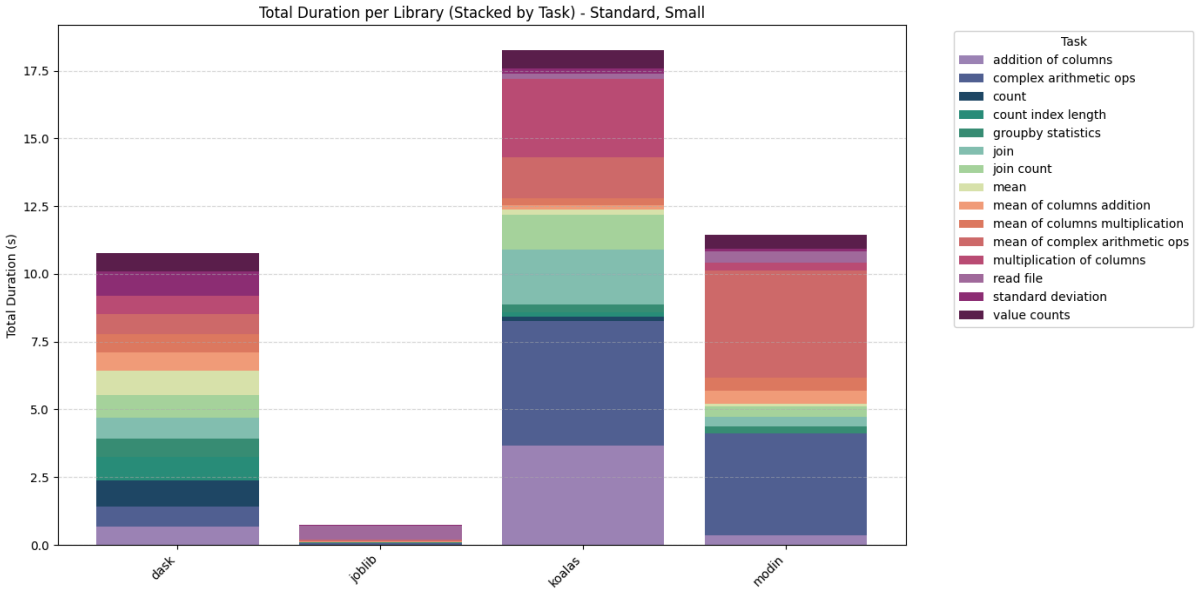
This graphic is a stacked bar chart representing the total execution time spent by different libraries on various tasks for a specific combination of dataset variant and size.

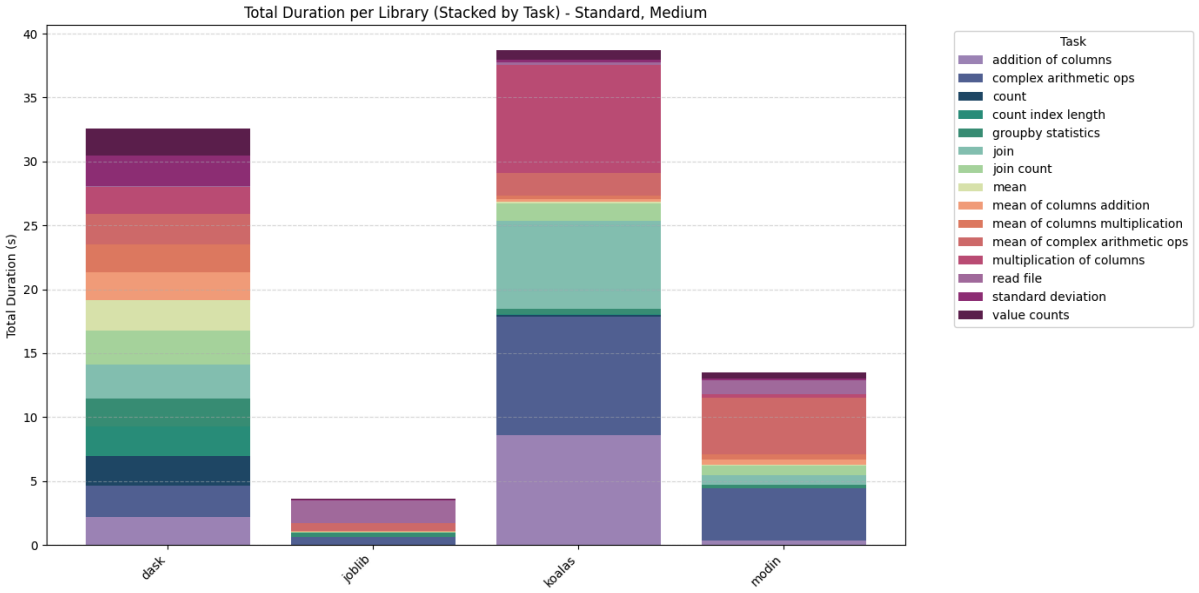
For each library, the total duration of each individual task is stacked on top of one another, showing their contribution to the overall execution time.

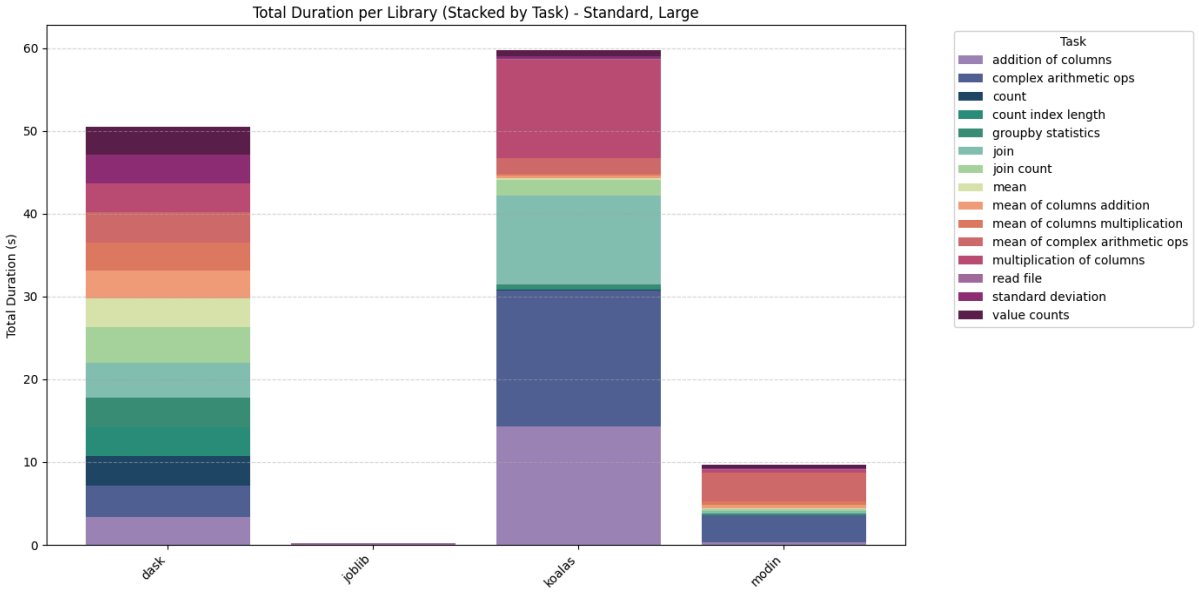
- The x-axis lists the libraries, such as Koalas, Dask, Modin, Joblib.
- The y-axis shows the total execution time in seconds.
- Each color segment within a bar corresponds to a different task, allowing you to see which tasks dominate the runtime for each library.
- Separate plots are generated for each pair of variant (e.g., standard, filtered, cached) and dataset size (small, medium, large), enabling detailed comparison across these dimensions.

This visualization helps identify which libraries perform better or worse for particular dataset conditions and which tasks are the main contributors to their runtime.

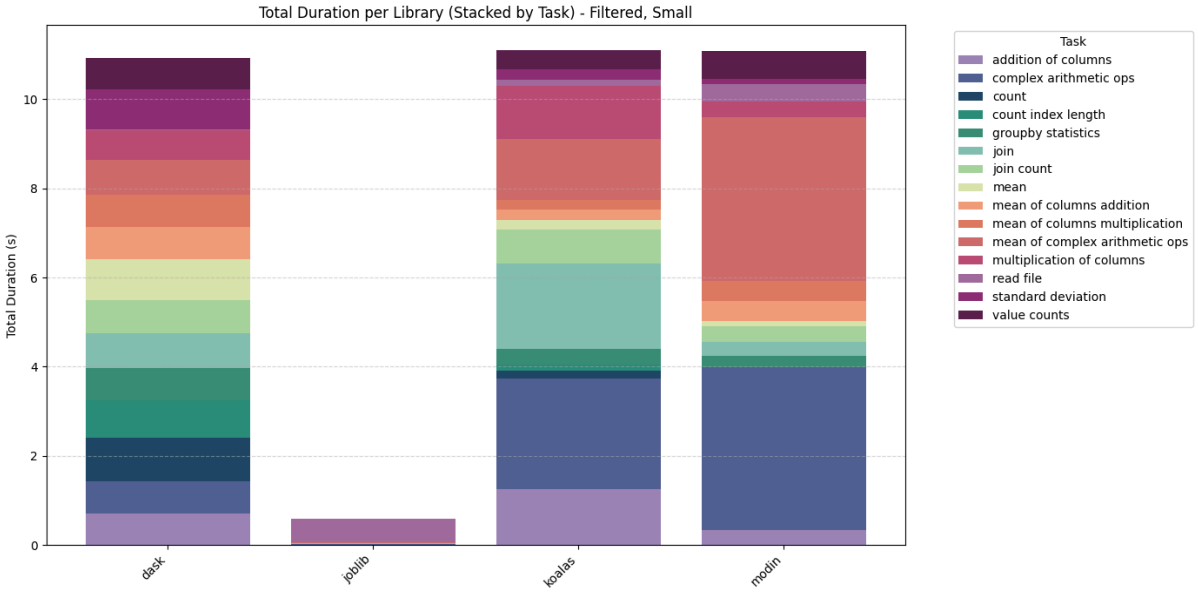
1. Standard Datasets (Small, Medium, Large)

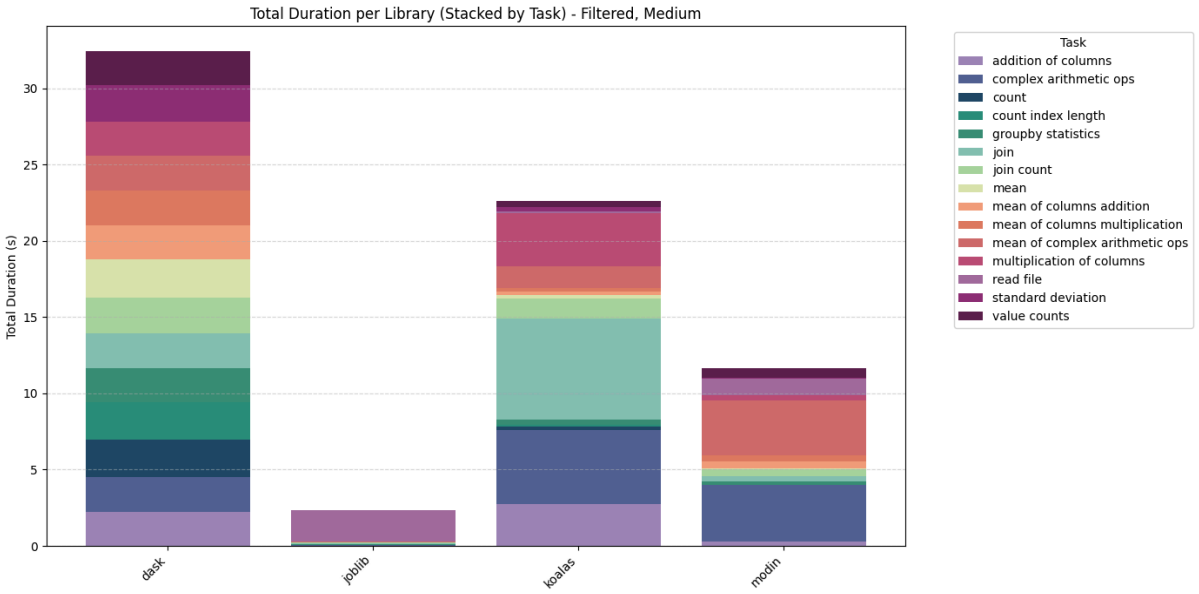


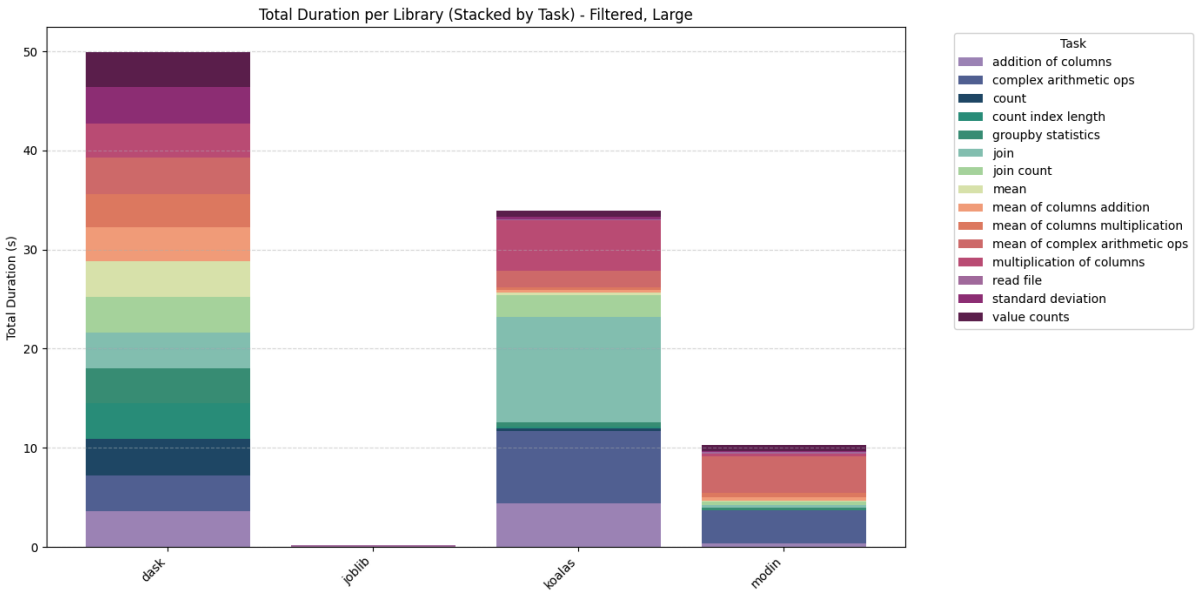




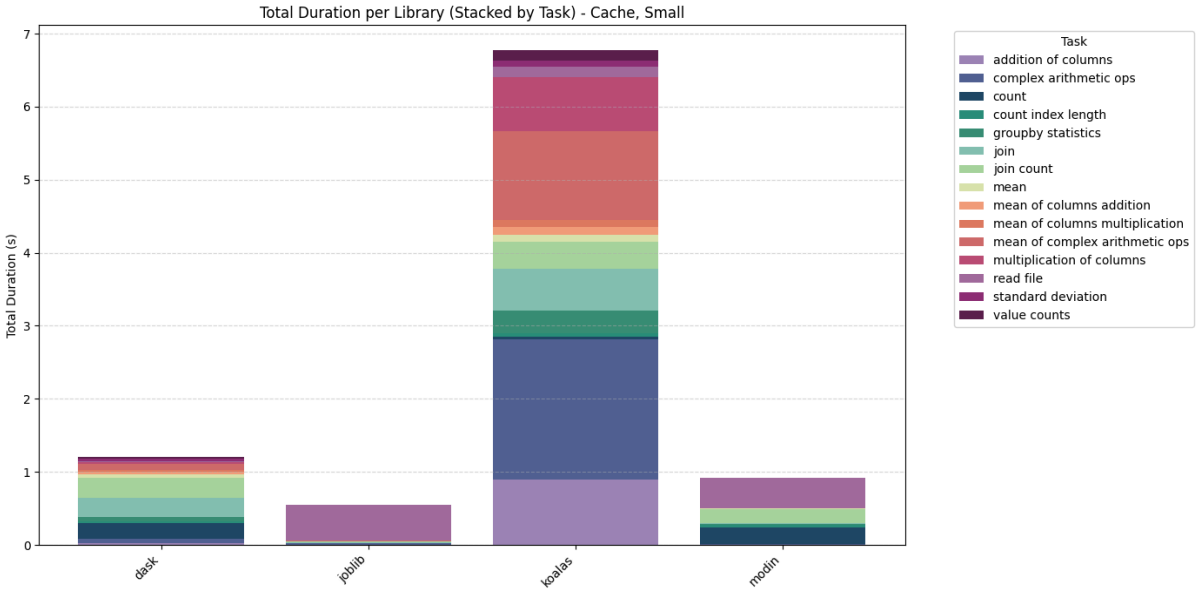
2. Filtered Datasets (Small, Medium, Large)

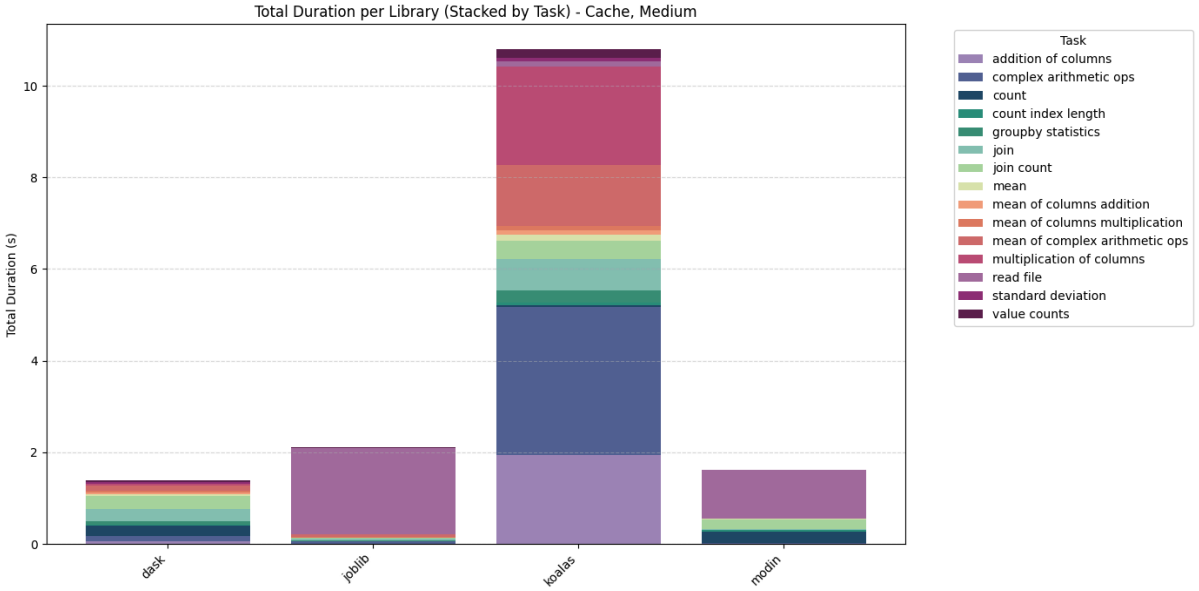


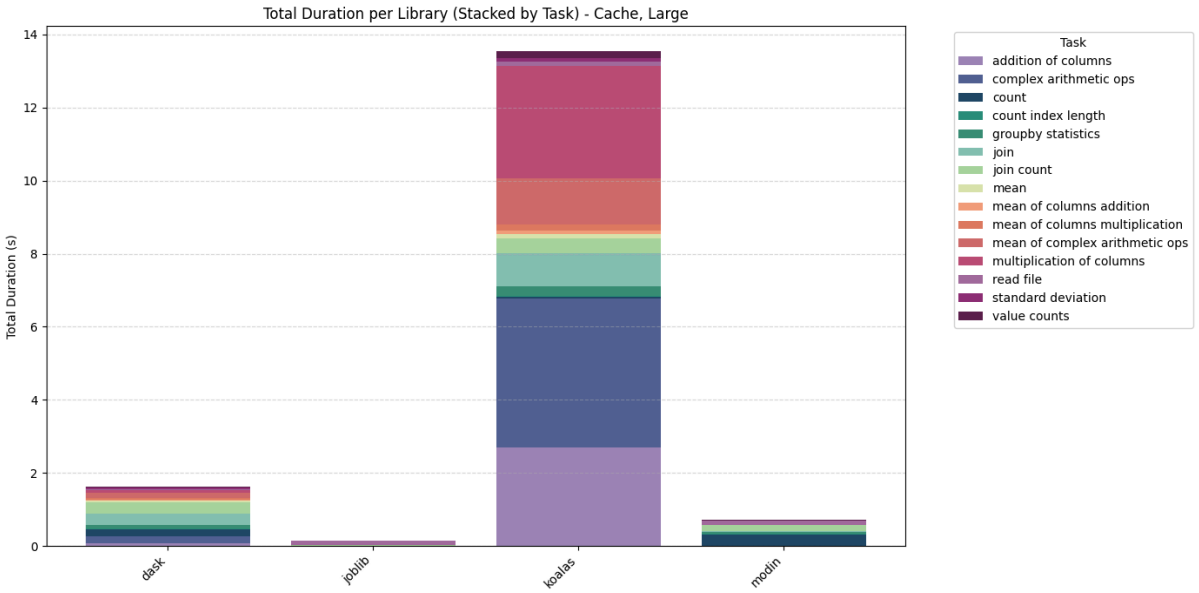




3. Cache Datasets (Small, Medium, Large)







Analysis

For the small and medium datasets, the libraries with the highest execution times vary depending on the dataset variant:

- In the **standard small** dataset and **standard medium** dataset, Koalas consistently shows the highest bars, reflecting its significant overhead due to Spark orchestration, which is noticeable even at smaller scales.
- In the **filtered small** dataset, the highest bars appear for Koalas, Dask, and Modin. This suggests that these libraries incur greater overhead when managing distributed or parallel execution on filtered data, where the dataset size and complexity do not justify their heavier coordination.
- For the **filtered medium** dataset, Dask bars become dominant, highlighting that Dask struggles with performance on filtered datasets of moderate to large size, likely due to task scheduling overhead and inefficient parallelization on these specific workloads.
- In the **cached medium** and **cached large** datasets, Koalas again shows the highest bars. This indicates that even with cached data—which removes input/output bottlenecks—Koalas's Spark-based backend introduces latency that negatively impacts total execution time compared to other libraries.

Justification:

The trend shows that Koalas tends to have the largest execution time overhead across most dataset sizes and variants because Spark's job scheduling and orchestration costs remain significant regardless of caching or filtering. Dask, while designed for parallelism, shows performance degradation on filtered datasets, likely due to task management overhead that is not compensated by parallel speedup at those scales. Modin also experiences overhead on filtered datasets, but to a lesser extent than Dask or Koalas. On the other hand, libraries like Joblib maintain relatively low execution times, as they operate with minimal overhead and favor in-memory computations.

Rapids comparison

The following graphics are a group of stacked bar charts showing the total execution time spent on various tasks using the RAPIDS framework.

Each plot corresponds to a specific combination of dataset variant (standard, filtered, cached) and dataset size (small, medium, large).

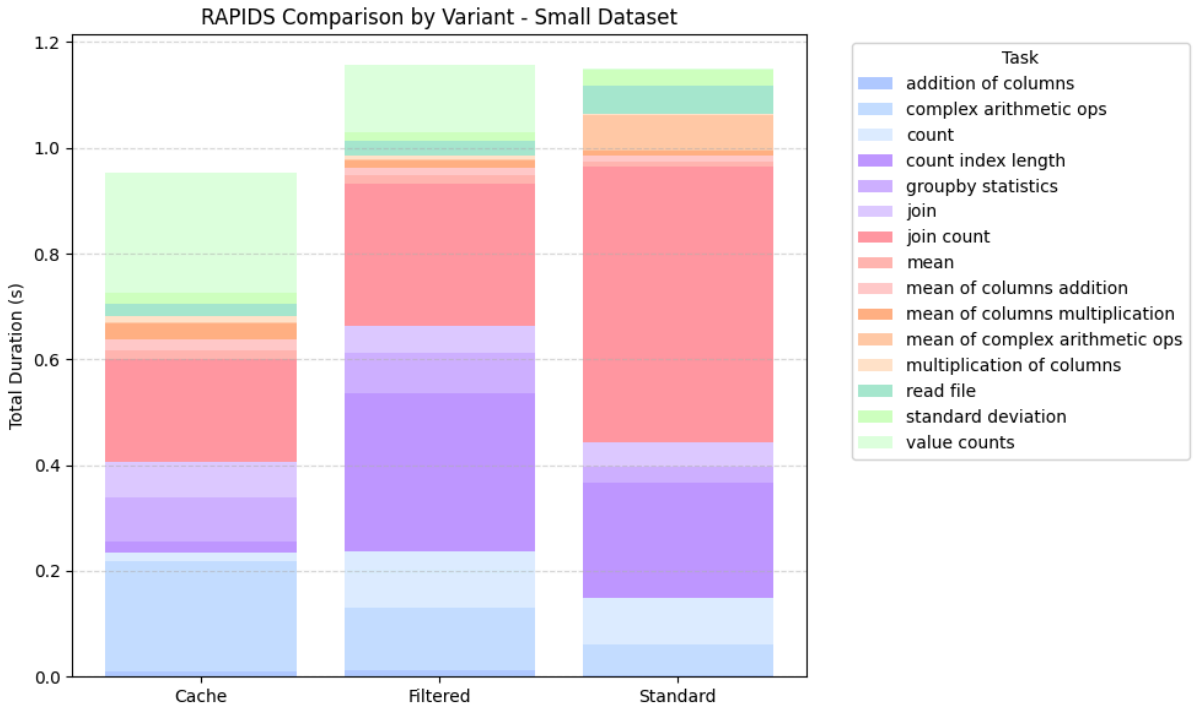
- The x-axis displays the variants, while the y-axis shows the total execution time in seconds.

- Each colored segment within a bar represents a different task, making it easy to identify which tasks contribute most to the total runtime.

The chart allows comparison of task performance across different data volumes and processing strategies. This visualization also helps highlight the main bottlenecks depending on the variant and dataset size.

It is designed in order to compare RAPIDS performance under different conditions.

1. **Small Dataset**



Analysis: Standard > Filtered > Cache

- **Standard (slowest):**

In small datasets, orchestration/setup overhead becomes disproportionately large compared to the actual computation. RAPIDS still launches GPU operations, which include memory transfers and kernel launches — these introduce fixed costs even when data is small. In the standard variant, all operations are applied, making this overhead most visible.

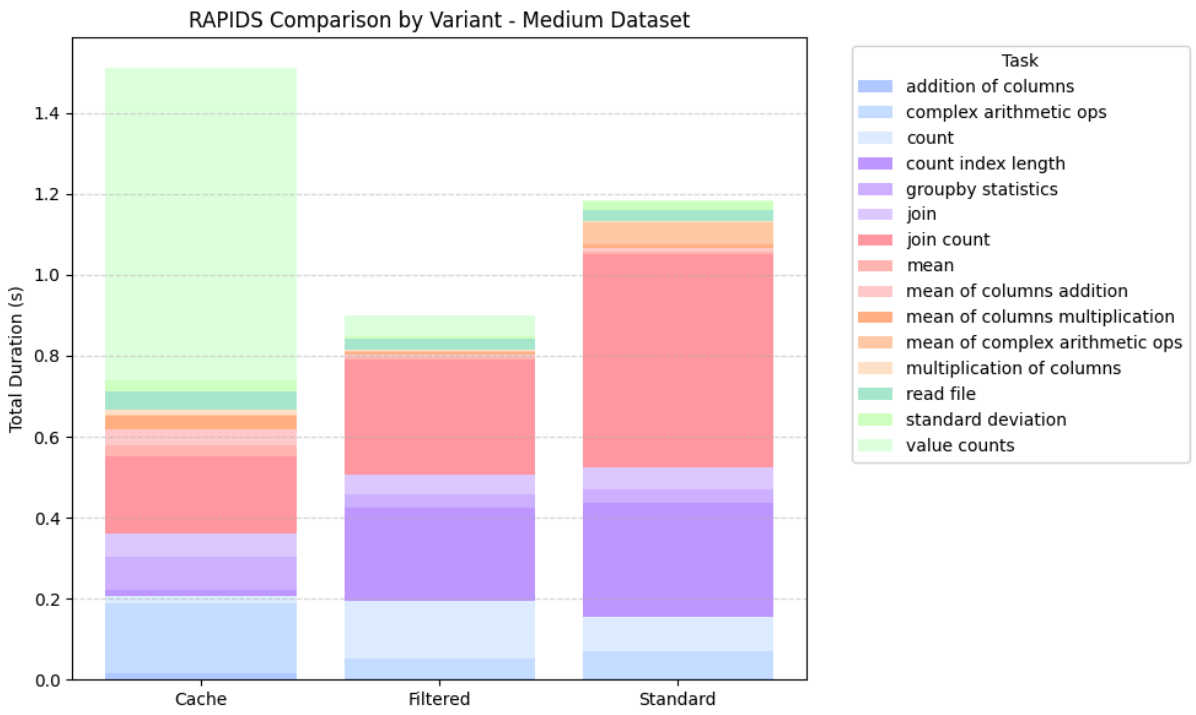
- **Filtered (faster):**

The filtered variant likely includes fewer operations or operates on fewer rows/columns, reducing the amount of data transferred to the GPU and the number of GPU operations required. Hence, execution time drops.

- **Cache (fastest):**

The cache variant benefits from data already being loaded in GPU memory or from optimizations due to repeated access. This drastically reduces I/O and transfer overhead, making it the fastest.

2. Medium Dataset



Analysis: Cache > Standard > Filtered

- **Cache (slowest):**

With a medium-sized dataset, caching the entire dataset into GPU memory can stress memory capacity or lead to inefficient memory management. Data may not fit neatly in memory, causing

paging or delays. Also, operations that benefit from lazy evaluation (in standard) may now be forced to materialize eagerly in cache.

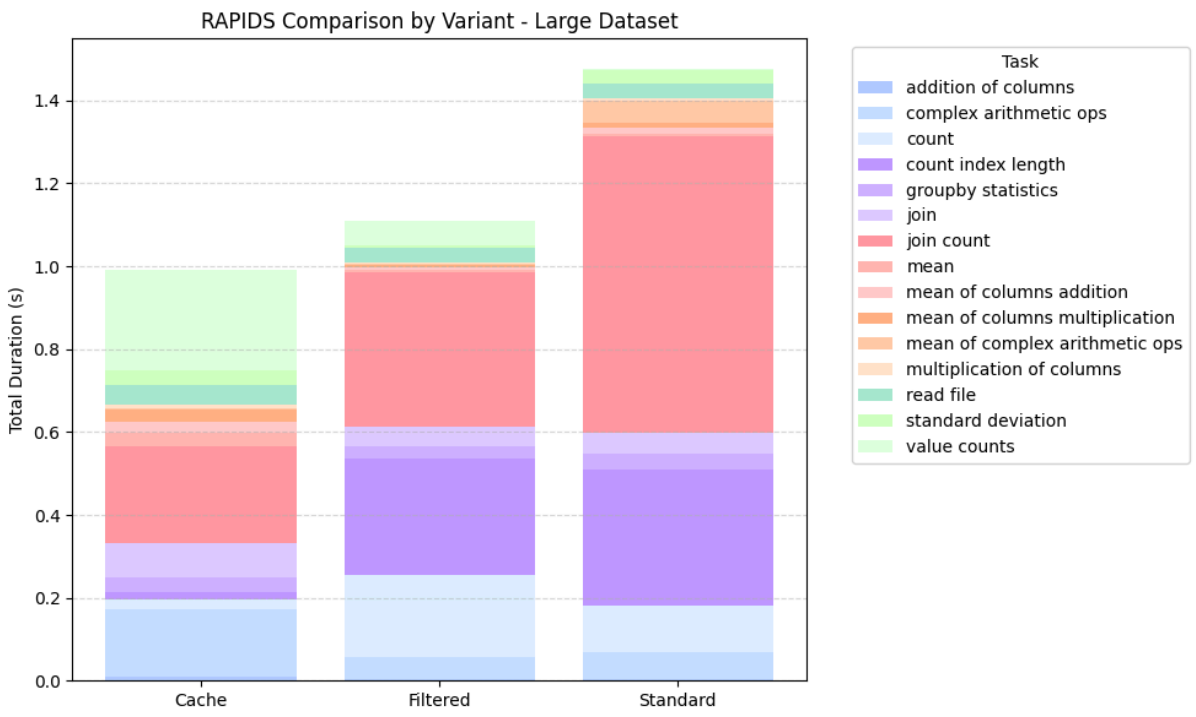
- **Standard (middle):**

At this size, RAPIDS performs well when leveraging its normal workflow — parsing data and applying operations without forcing everything into GPU memory. It balances overhead with GPU acceleration.

- **Filtered (fastest):**

Since filtered datasets are lighter, the GPU is used more efficiently. Fewer operations and less data make RAPIDS very effective, minimizing memory pressure and kernel launch overhead.

3. Large Dataset



Analysis: Standard > Filtered > Cache

- **Standard (slowest):**

In the standard variant, RAPIDS has to process all data and all tasks, possibly pushing GPU memory limits. This causes slower performance due to memory overuse or kernel queuing.

- **Filtered (faster):**

Filtered datasets contain less data, so RAPIDS can process them more efficiently. Even with large size, the workload is smaller than in the standard variant.

- **Cache (fastest):**

If caching is properly managed and the data fits into GPU memory, cached large datasets allow RAPIDS to skip repeated I/O, improving execution time significantly. It avoids data transfer overhead and makes full use of GPU acceleration for all tasks.

4. Machine Learning Analysis

4.1. Methodology

Data Preprocessing and Feature Engineering

In this stage, a comprehensive data preprocessing pipeline was applied to ensure the quality, consistency, and suitability of the data for further analysis and modeling. The main steps are described below:

- **Datetime Formatting and Filtering:**
  The columns **Trip_Pickup_DateTime** and **Trip_Dropoff_DateTime** were converted to proper datetime objects. The dataset was then filtered to include only trips from the years 2009 and 2010, aligning with the scope of the analysis.

- **Column Cleaning:**
  Irrelevant columns such as **index** and **pickup_date** were removed to reduce redundancy.

- **Payment Type Normalization:**
  Various inconsistent values in the **Payment_Type** field were standardized using a custom mapping strategy. This ensured that all payment types were grouped under consistent labels like **CASH, CREDIT, NO_CHARGE,** and **DISPUTE**.
  One-hot encoding was applied to both **Payment_Type** and **vendor_name** to enable their use in machine learning models.

- **New Feature Construction**:
  Several derived features were introduced:
  - **trip_duration_sec**: the total duration of each trip in seconds.
  - **Fare_per_mile:** the fare normalized by the trip distance.

- **Tip_percent:** the tip amount as a proportion of the total fare.

- **Spatial Clustering (Pickup Zones):**
  To capture spatial behavior, clustering of pickup coordinates was performed using HDBSCAN, a density-based clustering algorithm well-suited for large-scale geospatial data. The algorithm automatically inferred the number of spatial clusters (zones) without requiring manual specification.

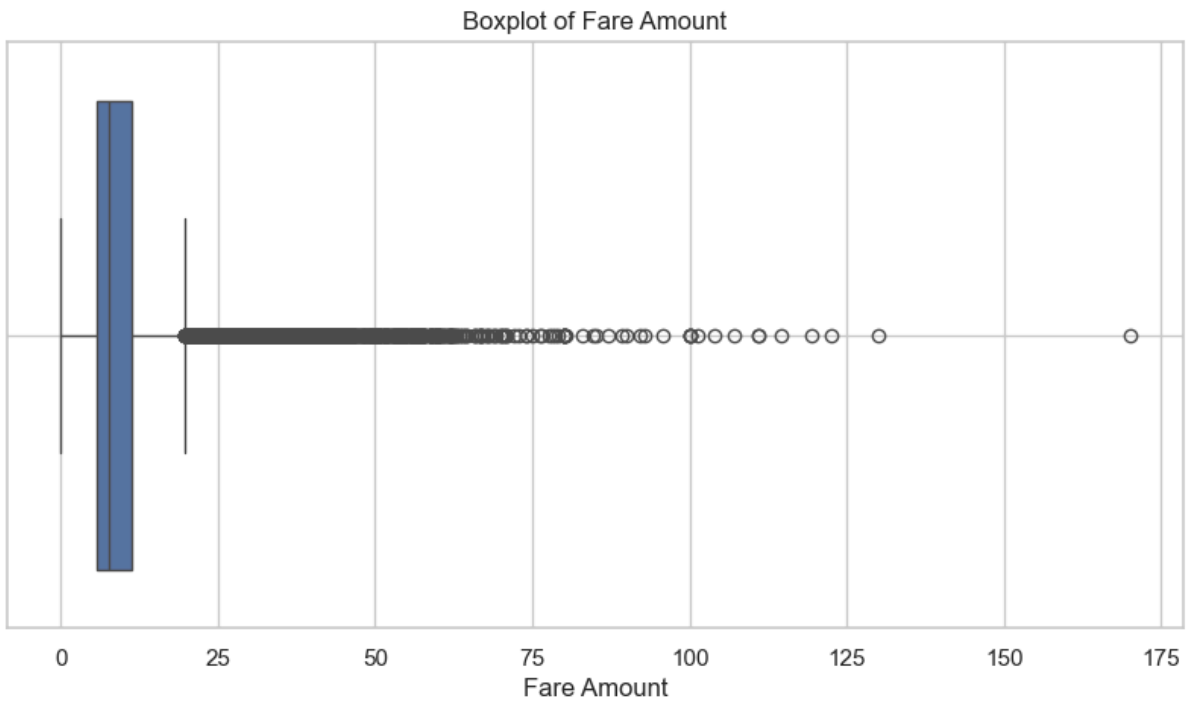
- **Temporal Feature Extraction:**
  Temporal information was extracted from the pickup and dropoff timestamps, including:
  - **Hour**, **day_of_the_week**, **month**, and **year** of both pickup and dropoff events. These features support the identification of time-based patterns and seasonality effects.

- **Data Cleaning and Outlier Removal:**
  To eliminate noise and ensure reliability:
  - Trips with zero passengers or zero distance were removed.
  - Trips with negative fare amounts were excluded.

- **Final Column Cleanup:**
  After extracting useful temporal information, original datetime columns and unused temporal features (**year_month, day**) were dropped to avoid redundancy.

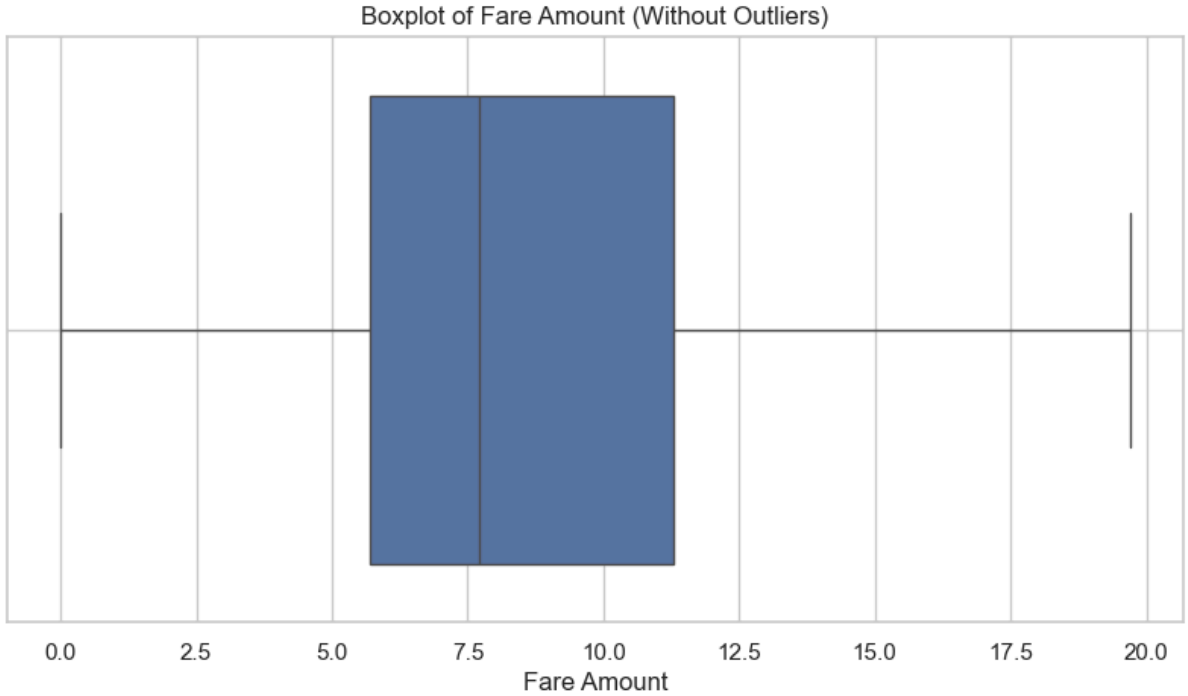
This structured preprocessing workflow ensured that the dataset was clean, consistent, and enriched with meaningful features, enabling more effective modeling and analysis in subsequent phases.

Understanding the Target



The boxplot of the target variable Fare_Amt clearly reveals a significant number of outliers. While the majority of fare values are concentrated within a reasonable range, there are numerous extreme values extending far beyond the upper whisker of the boxplot.

These high-fare outliers can heavily skew the distribution and potentially impact model performance if not handled appropriately.



When these outliers are removed, the range of the boxplot reduces drastically, indicating that most fare values are actually concentrated within a much narrower and more typical interval.

This suggests that the distribution of fares is heavily skewed by a relatively small number of extreme values, reinforcing the importance of careful outlier treatment in the modeling pipeline.

Machine Learning Models

XGBRegressor

The **XGBRegressor** is a powerful and efficient implementation of gradient boosted decision trees, provided by the XGBoost library. It is designed for high performance and scalability, making it a popular choice for regression tasks, particularly when working with large structured datasets. Its strength lies in its ability to model complex nonlinear relationships while offering built-in regularization (*lambda* and *alpha*) to prevent overfitting.

In our experiment, we used XGBRegressor to predict taxi fare amounts (**Fare_Amt**) based on various trip attributes.

We applied a **10-fold cross-validation** strategy using **KFold** from **Scikit-learn** to ensure robust performance estimation. Each fold involved training the model on 90% of the data and validating on the remaining 10%. The model was instantiated with carefully chosen hyperparameters:

- **n_estimators=100:** Limits the number of boosting rounds to prevent overfitting.
- **max_depth=10:** Controls the depth of each individual tree, balancing model complexity and generalization.
- **learning_rate=0.1:** Scales the contribution of each tree to the ensemble; lower values provide more gradual learning.
- **random_state=42:** Ensures reproducibility.
- **n_jobs=-1:** Enables parallel computation across all CPU cores for improved efficiency.

Each fold measured the model's performance using three key metrics: Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and R^2 score.

This cross-validation approach provided a reliable estimate of how well the model generalizes across different subsets of the data. We use **Joblib** internally by setting n_jobs=-1 in **XGBRegressor** to enable parallel computation across all CPU cores, speeding up model training and prediction without changing the output.

LogisticRegression

To evaluate the predictive capability of a linear classifier on taxi trip data, we used **LogisticRegression** as the primary supervised learning algorithm. Since the original target variable (**Fare_Amt**) is continuous, we first transformed it into a classification problem by applying **HDBSCAN**, an unsupervised density-based clustering algorithm. This allowed us to automatically derive meaningful class labels based on fare patterns and trip characteristics, effectively discretizing the regression target into categorical groups.

We selected key trip attributes — **Fare_Amt**, **Trip_Distance**, and **trip_duration_sec** — as inputs to the clustering algorithm. Features were standardized prior to clustering to ensure uniform scaling.This process yielded a multi-class target variable Fare_class, which captures natural groupings of fare behavior. Samples identified as noise (-1) were discarded to maintain label clarity.

With the target variable prepared, we trained a LogisticRegression classifier using the remaining trip features (excluding Fare_Amt and Fare_class) as predictors.

The classifier was configured with:

- **max_iter=2500:** to allow enough iterations for convergence, especially on larger and potentially imbalanced datasets.
- **n_jobs=-1:** to leverage all available CPU cores through parallelism.
- **random_state=42:** to ensure deterministic behavior across folds and reproducibility.

We used **Stratified K-Fold Cross-Validation** with **10 folds** to maintain consistent label distribution across train and test sets, which is critical for multi-class classification.

To fairly assess performance across all classes, we computed macro-averaged metrics—including **accuracy**, **precision**, **recall**, and **F1-score** — across all folds. This ensured that each class, regardless of its frequency, contributed equally to the evaluation.

We use **Joblib** internally by setting n_jobs=-1 in **LogisticRegression** to enable parallel computation across all CPU cores, speeding up model training and prediction without changing the output.

5. Conclusion

This project extended beyond tool benchmarking—it surfaced critical patterns in how big data libraries scale, behave under memory pressure, and interact with I/O at different stages of a typical machine learning pipeline.

By evaluating Koalas, Dask, Modin, Joblib, and Rapids across three dataset configurations — standard, filtered, and cached — we found **performance is not merely a function of library design, but of how well that design aligns with workload characteristics**.

Koalas, despite its distributed Spark backend, consistently struggled due to orchestration overhead and latency from lazy execution — even with cached datasets. Its design shines in massive, distributed clusters, but proved inefficient for medium-scale interactive workloads.

Dask displayed strong scalability but was hampered in filtered and smaller datasets by the coordination cost of its task graph. It benefits most when parallelism clearly outweighs overhead, such as in large, memory-bound tasks.

Modin excelled in pandas-like syntax and offered decent performance in cache-friendly operations. However, it often fell short in I/O-heavy tasks and more complex parallel computations, pointing to limitations in abstracting pandas APIs to distributed backends.

Joblib emerged as the most consistent performer across all scenarios. Its focus on **in-memory parallelism using shared memory and threading/multiprocessing** proved ideal for mid-sized data and compute-heavy operations, where minimal orchestration was a virtue rather than a constraint.

Rapids, though not evaluated across all phases, showcased the **future-facing potential of GPU acceleration**—especially for compute-bound tasks — provided the system is tuned and provisioned appropriately.

From a modeling perspective, both regression and classification pipelines added valuable insight. **XGBoost Regressor**, with optimized hyperparameters and CPU-parallelized training (n_jobs=-1), handled high-dimensional fare prediction robustly. Meanwhile, we leveraged **HDBSCAN to transform a continuous target into meaningful classes**, enabling **Logistic Regression** to perform interpretable classification on derived fare tiers — demonstrating how unsupervised and supervised methods can be combined to extract deeper structure from raw data.

Insights Beyond the Benchmark

There is no universally best tool — only tools best aligned to the problem's scale, complexity, and context. While distributed libraries promise scalability, they introduce coordination costs that can outweigh benefits on smaller data. Simpler, efficient tools like Joblib dominate in these scenarios. The key is **matching the library to the workload**, and designing pipelines that are both flexible and resource-aware.

This study reinforces the idea that **performance tuning isn't just technical — it's architectural**. The most effective data science workflows will be those that remain adaptable across varying data volumes, system constraints, and analysis goals.